定期扫描侵权店铺和违规分享(小红S,xian鱼等)

【拯救者】SSM期末速成(基础+期末题+文档) Java EE速成

可用于 期末

```
SSM =  Spring  +  SpringMVC  + Mybatis
```

👴少爷 👠公主请听课

【拯救者】Ep01_前言(必看 必看)

这里讲的是【 🌷速成🌷 速成🌷 速成】版本，按课本章节来， 抽取重点，翻译为大话！！！ 💝

🌻 适用于所有 SSM课本 🌻 适用于所有 SSM课本 🌻 适用于所有 SSM课本



学习本门课程，**无需课本， 下载下这个文档**，最后**会解题即可**！**遇到不会的**，**翻看文档**如何去破局！

【拯救者】SSM期末速成

🌀学习步骤：

每一章，都会【**讲会考的基础**💻】，**每章结束 会有【期末套题**🖍】讲解，

最后【**下载文档**📕】复习,即可 **拿捏考试**

🌻🌻🌻好评：



🌀个人简介：

前 阿里巴巴技术专家，【拯救者】系列课程广受好评，讲课风趣幽默

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)



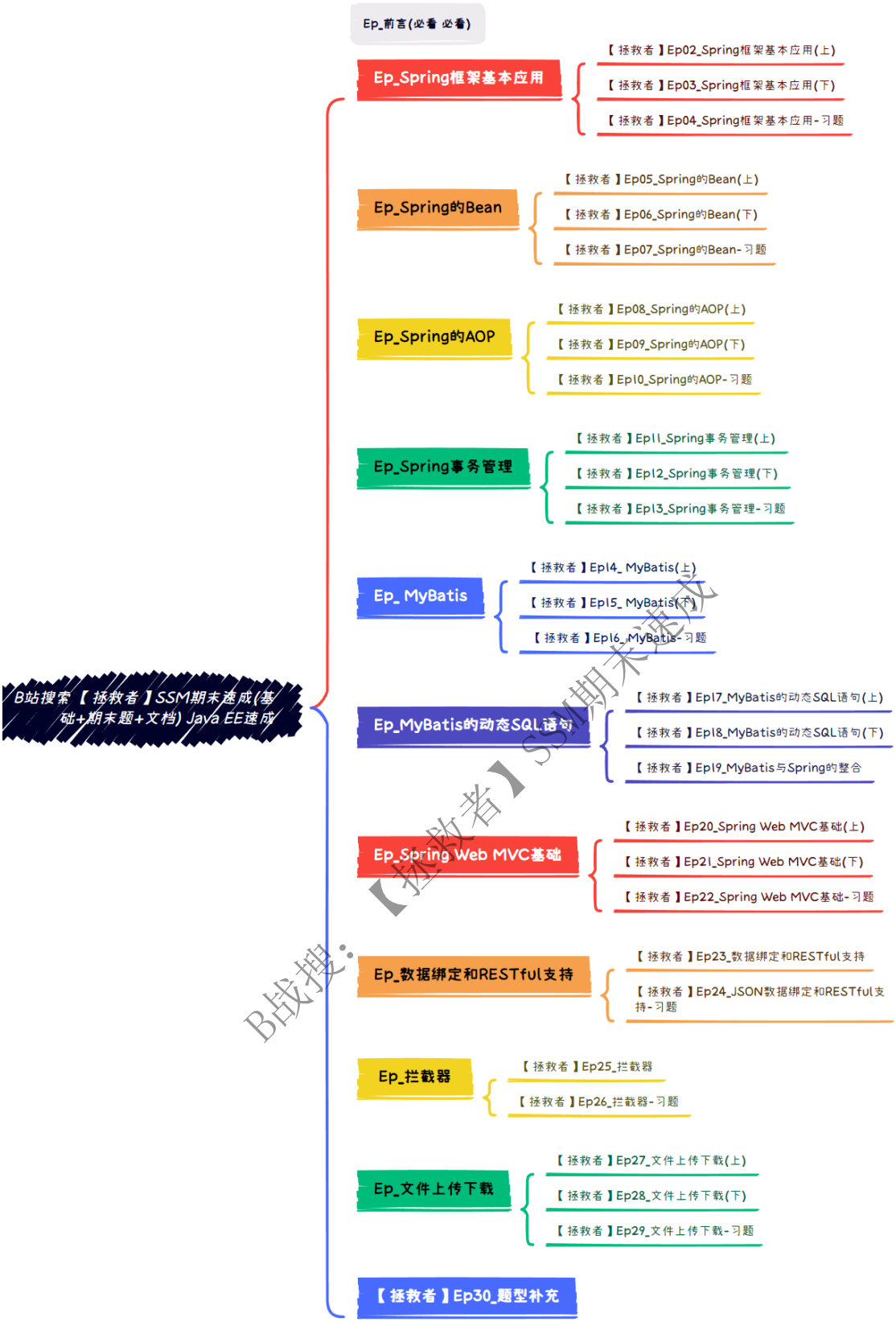


大纲:

B战搜：【拯救者】SSM期末速成

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)



B战搜:【拯救者】SSM期末速成

文档下载:

请看最后一集哈，**文档**都**给大家提供**的哦 **宝们**，**下载**完记得打开**【大纲模式】**（这样就会跟我一样有目录了）

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

不会打开大纲模式?

教程: https://mp.weixin.qq.com/s?__biz=MzkzNjYyODUzNQ==&mid=2247483718&idx=1&sn=01f6d1712bce71ccdac656f89820b1e1

开始大家不是很熟悉， 别着急

当你**二刷时**， 只会**感叹**:

【回首看，已如古人】~



让我们一起**出征【SSM速成】吧** ，爱谁谁，都多余了哦~~~

我们自己根据考点总结目录，无需看课本~

【拯救者】Ep02_Spring框架基本应用(上)

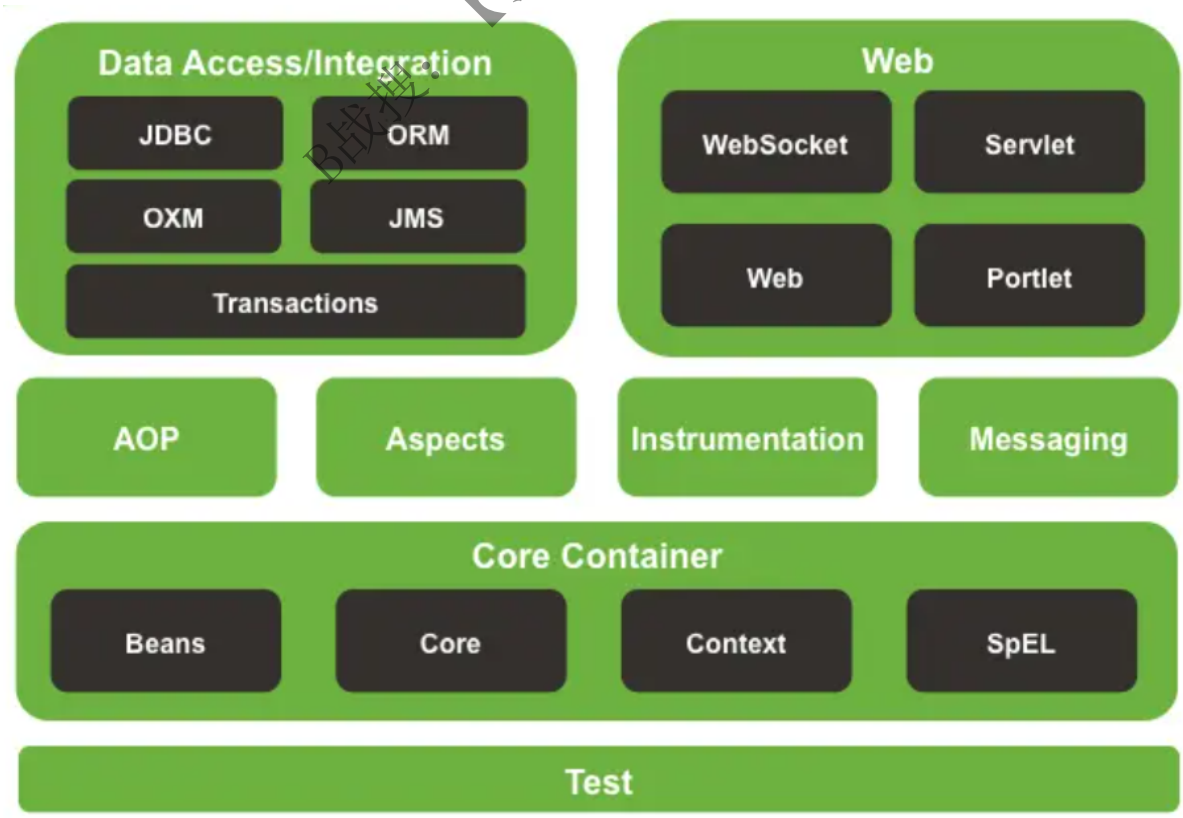
先讲Spring框架，然后讲MyBatis框架，最后讲Spring MVC框架

Spring简介

Spring 是个java企业级应用的开源开发框架。Spring主要用来开发web应用 .

是一个轻量级的控制反转 (IOC) 和面向切面 (AOP) 的容器框架

Spring体系结构



核心容器:

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

Core模块
    提供了框架的基本组成部分，包括IOC和依赖注入功能
Beans模块
    提供 BeanFactory，它是一个工厂模式的复杂实现
Context模块
     它是访问定义和配置的任何对象的媒介。ApplicationContext接口是其重点
SpEL模块
     表达式语言

Web层

Web模块
    提供了基本的面向web的集成功能，例如多个文件上传的功能和使用servlet监听器和初始化IOC容器
Web-MVC模块
    包含Spring的模型-视图-控制器(MVC)，实现了web应用程序
WebSocket模块
    为WebSocket-based提供了支持，而且在web应用程序中提供了客户端和服务器端之间通信的两种方式
Portlet模块
提供了在portlet环境中实现MVC

数据访问/集成

JDBC模块
    提供了删除冗余的JDBC相关编码的JDBC抽象层
ORM模块
    为流行的对象关系映射API，包括JPA， JDO,Hibernate和iBatis,提供了集成层
OXM模块
    提供了抽象层，它支持对JAXB,Castor,XMLBeans，JiBX和XStream 的对象/XML映射实现
Transactions模块
    为实现特殊接口的类及所有的POJO支持编程式和声明式事务管理
JMS模块
    Java消息服务，包含生产和消费的信息的功能

其他模块

AOP模块
    提供了面向切面的编程实现，允许你定义方法拦截器和切入点
Aspects模块
    提供了与AspectJ的集成，这是一个功能强大且成熟的面向切面
Messaging模块
   处理来自WebSocket客户端的信息

Spring特征

三大特征：**AOP、IOC、DI。**

IOC ： 控制反转。
DI ： 依赖注入。
AOP ： 面向切面编程。

B战搜：【拯救者】SSM期末速成

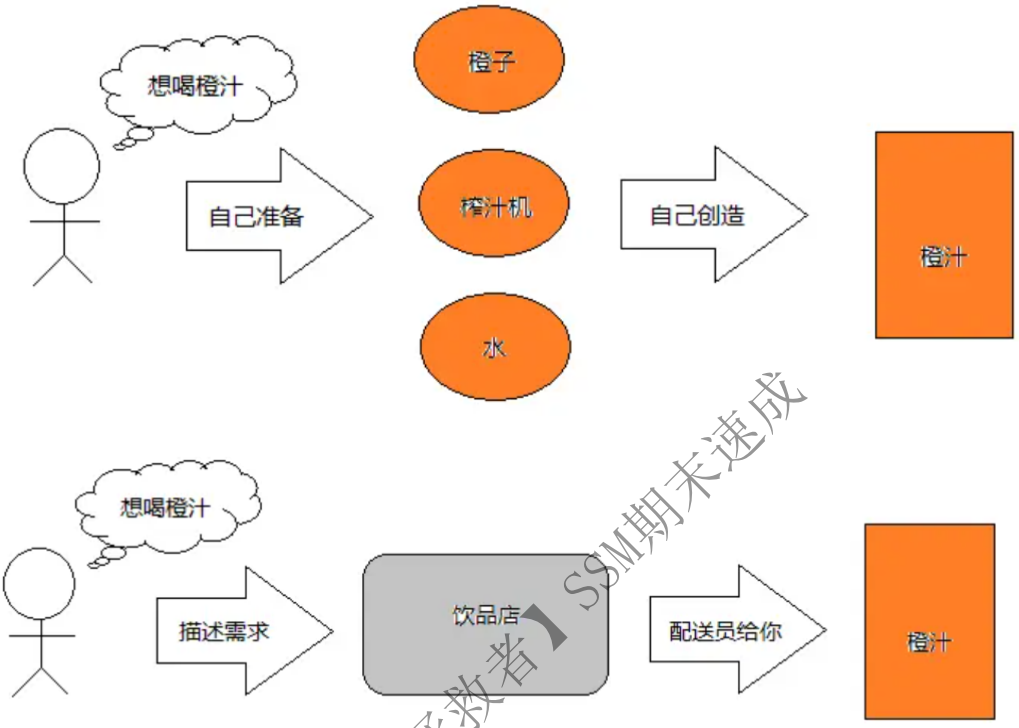
定期扫描侵权店铺和违规分享(小红S,xian鱼等)

Spring IoC/DI概述

IOC（控制反转）

- 使用目的：降低耦合度

> 正向控制：若要使用某个对象，需要自己去负责对象的创建。
> 反向控制：若要使用某个对象，只需要从 Spring 容器中获取需要使用的对象，不关心对象的创建过程，也就是把创建对象的控制权反转给了Spring框架。



B战搜：【拯救者】SSM期末速成

DI的方式(依赖注入)

> 对象只提供普通的方法，容器全权负责组件的装配，它会把符合依赖关系的对象通过属性（JavaBean中的setter）或者是构造 传递给需要的对象。

Spring IoC容器

IOC容器负责对象的创建、初始化等一系列工作，**被创建或被管理的对象在IOC容器中统称为Bean。**

> 常见的IoC容器实现方式包括 BeanFactory和ApplicationContext。
> （1）BeanFactory：Spring内部使用的接口，不提倡开发人员使用。特点：加载配置文件时不会创建对象，获取对象时才会创建对象。
> （2）ApplicationContext：BeanFactory的子接口，提供了更多更强大的功能，一般由开发人员使用。特点：加载配置文件时会把配置文件里的对象进行创建。

Spring框架入门案例

引入依赖:

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
    <version>5.3.28</version>
  </dependency>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-beans</artifactId>
    <version>5.3.28</version>
  </dependency>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <version>5.3.28</version>
  </dependency>
  <dependency>
    <groupId>commons-logging</groupId>
    <artifactId>commons-logging</artifactId>
    <version>1.2</version>
  </dependency>
  <dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid</artifactId>
    <version>1.2.8</version>
  </dependency>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>5.3.28</version>
  </dependency>
```

创建Book类

三种注入属性得到方式

```
第一种：set方法注入          （本节讲）
第二种：有参构造方法注入（下节讲）
第仨种：p名称空间注入        （下节讲）
```

```
public class Book {
    private String bookName;
    private double price  ;

    public void setBookName(String bookName) {
        this.bookName = bookName;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public void show(){
        System.out.println(bookName+":"+price);
    }
}
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

创建配置文件 applicationContext.xml

```
<bean id="book" class="com.wuji.Book"></bean>
```

配置文件中通过property标签进行属性注入

```
<bean id="book" class="com.wuji.Book">
    <property name="bookName" value="我是张无忌"></property>
    <property name="price" value="199.9"></property>
</bean>
```

main方法测试

```
 <!--加载spring配置文件创建对象-->
 ApplicationContext context=new
ClassPathXmlApplicationContext("applicationContext.xml");
 Book book=context.getBean("book", Book.class);
 book.show();
```

【拯救者】Ep03_Spring框架基本应用(下)

第二种：有参构造方法注入

```
public class Book {
    private String bookName;
    private double price  ;

    public Book(String bookName,double price) {
        this.bookName = bookName;
         this.price = price;
    }

    public void show(){
        System.out.println(bookName+":"+price);
    }
}
```

```
<bean id="book" class="com.wuji.Book">
          <constructor-arg name="bookName" value="我是张无忌"></constructor-arg>
          <constructor-arg name="price" value="199.9"></constructor-arg>
</bean>
```

第仨种：p名称空间注入

使用p:属性名 = 赋予属性的值

B战搜:【拯救者】SSM期末速成

B战搜: 【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
 <!--添加p名称空间-->
       xmlns:p="http://www.springframework.org/schema/p"

    <bean id="book" class="com.wuji.Book" p:bookName="我是张无忌" p:price="199.9">
</bean>
```

也可以使用**FileSystemXmlApplicationContext**

```
ClassPathXmlApplicationContext  默认情况下路径是不需要加classpath:
FileSystemXmlApplicationContext 读取绝对路径 需要加上classpath:
```

```
 ApplicationContext actx = new
FileSystemXmlApplicationContext("src/main/java/applicationContext2.xml");

 ApplicationContext actx2= new
FileSystemXmlApplicationContext("classpath:applicationContext2.xml");
```

【拯救者】Ep04_Spring框架基本应用-习题

依赖注入的作用就是在使用 Spring 框架创建对象时，动态的将其所依赖的对象注入到【 】组件中。

B战搜:【拯救者】SSM期末速成

B战搜: 【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

在使用 Spring 框架之后，对象的实例不再由调用者来创建，而是由【　　】来创建，Spring 容器会负责控制程序之间的关系，而不是由调用者的程序代码直接控制。

以下关于 Spring 核心容器相关说法错误的是。（　）

A: Spring 框架的所有功能都是通过其核心容器来实现的。

B: 创建 BeanFactory 实例时，需要提供 Spring 所管理容器的详细配置信息，这些信息通常采用 XML 文件形式来管理。

C: ApplicationContext 不仅包含了 BeanFactory 的所有功能，还添加了对国际化、资源访问、事件传播等方面的支持。

D: 通常在 Java 项目中，会采用通过 ClassPathXmlApplicationContext 类来实例化 ApplicationContext 容器的方式，而在 Web 项目中，ApplicationContext 容器的实例化工作会交由 Web 服务器来完成。

以下有关 Spring 的四个基础包说法正确的是。（　）

A: Spring 的四个基础包，它们分别对应 Spring Web 容器的四个模块。

B: Spring 的四个基础包有 spring-core.RELEASE.jar 、spring-beans-.RELEASE.jar 、spring-context-.RELEASE.jar 和 spring-aop-.RELEASE.jar 。

C: spring-context-.RELEASE.jar 是所有应用都要用到的 JAR 包，它包含访问配置文件以及进行 IoC 或者 DI 操作相关的所有类。

D: spring-core.RELEASE.jar 包含 Spring 框架基本的核心工具类，Spring 其它组件都要用到这个包里的类，是其它组件的基本核心。

以下有关 Spring 框架优点的说法正确的是。（　）

A: Spring 具有简单、可测试和松耦合等特点，从这个角度出发，Spring 就是应用于任何 Java 应用的开发中；

B: Spring 提供了对 AOP 的支持，它允许将一些通用任务，如安全、事务、日志等进行集中式处理，从而提高了程序的复用性

C: Spring 就是一个大工厂，可以将所有对象的创建和依赖关系的维护工作都交给 Spring 容器管理，杜绝了组件之间的耦合性

D: Spring 增加了 Java EE 开发中一些 API 的使用难度

Spring 的核心容器是其他模块建立的基础，以下哪个不是该容器的组成模块。（　）

A: Beans 模块

B: Core 模块

C: Context 模块

D: AOP 模块

下列选项中，不属于 Spring 框架优点的是。（　）

A: 提供强大的、可以有效减少页面代码的标签

B: 声明式事务的支持。

C: 方便解耦、简化开发

D: 方便集成各种优秀框架

B战搜：【拯救者】SSM期末速成

请看视频讲解

简答题

Spring 依赖注入通常有哪些实现方式，列举并分别进行说明。

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

Spring 的实现方式通常有两种， 一种是属性 setter 方法注入， 另一种是构造方法注入。

setter方法注入
是指 IoC 容器使用 setter 方法来注入被依赖的实例。 通过调用无参构造器或无参静态工厂方法实例化 Bean 后，调用该 Bean 的 setter 方法，即可实现基于 setter 方法的依赖注入。

构造方法注入
是指 IoC 容器使用构造方法来注入被依赖的实例。基于构造方法的依赖注入通过调用带参数的构造方法来实现，每个参数代表着一个依赖。

请简述Spring框架的四个基础包并说明作用。

spring-core-4.3.6.RELEASE.jar
包含Spring框架基本的核心工具类，Spring其它组件都要用到这个包里的类，是其它组件的基本核心。
spring-beans-4.3.6.RELEASE.jar
所有应用都要用到的JAR包，它包含访问配置文件、创建和管理Bean以及进行控制反转或者依赖注入操作相关的所有类。
spring-context-4.3.6.RELEASE.jar
Spring提供了在基础控制反转功能上的扩展服务，还提供了许多企业级服务的支持
spring-expression-4.3.6.RELEASE.jar
定义了Spring的表达式语言。

【拯救者】Ep05_Spring的Bean(上)

Bean作用域

singleton：单例模式：IOC容器中只会存在一个bean实例，无论多少bean引用它，始终指向相同的对象
prototype：原型模式：IOC容器可以同时存在多个实例，当程序去获取bean时可创建出一个新的实例
request：在一次Http请求中，容器会返回该Bean的同一实例。而对不同的Http请求则会产生新的Bean，而且该bean仅在当前Http Request内有效。
session：在一次Http Session中，容器会返回该Bean的同一实例。而对不同的Session请求则会创建新的实例，该bean实例仅在当前Session内有效。
global Session：在一个全局的Http Session中，容器会返回该Bean的同一个实例，仅在使用portlet context时有效。

singleton作用域

```
<bean ...   scope="singleton"></bean>
```

prototype作用域

```
<bean ...   scope="prototype"></bean>
```

Bean的自动装配

自动装配是使用Spring满足bean依赖的一种方法，Spring会在**应用上下文中**为某个bean**寻找其依赖的bean。**

新建两个实体类：Cat、Dog

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
    public class Cat {
      public void show() {
        System.out.println("miao~");
      }
    }
    public class Dog {
      public void show() {
        System.out.println("wang~");
      }
    }
```

新建一个Animal类

```
    public class Animal {
      private Cat cat;
      private Dog dog;
      set  get
    }
```

编写Spring配置文件。

```
  <bean id="dog" class="com.wuji.Dog"/>
  <bean id="cat" class="com.wuji.Cat"/>
  <bean id="animal" class="com.wuji.Animal">
     <property name="cat" ref="cat"/>
     <property name="dog" ref="dog"/>
  </bean>
```

测试

```
    public class MyTest {
      @Test
      public void testMethodAutowire() {
        ApplicationContext context = new
ClassPathXmlApplicationContext("xx.xml");
        Animal animal =   context.getBean("animal",Animal.class);
        animal.getCat().show();
        animal.getDog().show();
      }
    }
```

byName （按名称自动装配）

bean配置，增加一个属性autowire="byName"。

```
  <bean id="animal" class="com.wuji.Animal" autowire="byName">
  </bean>
```

测试结果依旧输出成功。

将cat的bean的id修改为其它的，如cat1，执行时报空指针java.lang.NullPointerException。

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
当一个bean节点带有autowire byName的属性时：
 将查找其类中所有的set方法名，例如setCat，获得将set去掉并且首字母小写的字符串，即cat。
    如果有，就取出注入；
    如果没有，就报空指针异常。
```

byType（按类型自动装配）

同一类型的对象, **要保证该类型在spring容器中唯一。如果不唯一，会报不唯一**

bean配置，改为autowire="byType"。

```
<bean id="animal" class="com.wuji.Animal" autowire="byType">
</bean>
```

再次测试结果依旧输出成功

再注册一个cat的bean对象,测试后报错：NoUniqueBeanDefinitionException。

```
    <bean id="cat2" class="com.wuji.Cat"/>
```

【拯救者】Ep06_Spring的Bean(下)

Spring2.5开始全面支持**利用注解的**方式**注入属性。**

```
①在spring配置文件中引入context文件头。
  加入   xmlns:context="http://www.springframework.org/schema/context"
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd"

②开启属性注解支持。
    <context:annotation-config/>
```

【拯救者】SSM期末速成
B战搜：

Bean的基于Annotation的装配

@Autowired

@Autowired是按**类型**自动转配的,不用编写set方法了

将 类中的set方法去掉，使用@Autowired注解。

```
    public class Animal {
       @Autowired
      private Cat cat;
       @Autowired
      private Dog dog;
        get
    }
```

配置文件内容。

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
    <context:annotation-config/>
    <bean id="dog" class="com.wuji.Dog"/>
    <bean id="cat" class="com.wuji.Cat"/>
    <bean id="animal" class="com.wuji.Animal" />
```

测试后成功输出结果

@Qualifier

@Qualifier则可以根据**byName的方式自动装配。**@Qualifier不能单独使用。

配置文件修改内容

```
     <bean id="dog1" class="com.wuji.Dog"/>
     <bean id="dog2" class="com.wuji.Dog"/>
     <bean id="cat1" class="com.wuji.Cat"/>
     <bean id="cat2" class="com.wuji.Cat"/>
```

没有加@Qualifier测试会直接报错。
在属性上添加Qualifier注解。

```
    @Autowired
    @Qualifier(value = "cat2")
    private Cat cat;
    @Autowired
    @Qualifier(value = "dog2")
    private Dog dog;
```

@Resource

先按该属性进行byName方式查找装配。再进行默认的byName方式进行装配。

如果不成功，则按byType的方式自动装配。

如果**还不成功则报异常。**

```
    @Resource(name = "cat2")
    private Cat cat;
    @Resource
    private Dog dog;
```

其他几个注解

```
@Componet 描述spring中的一个bean
@Repository 将数据访问层（Dao层）的类标识为Spring中的bean
@Service 将业务层（Service层）的类标识为Spring中的bean
@Controller 将控制层的类标识为Spring中的bean
```

UserDao接口

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
public interface UserDao {
    public void save();

}
```

UserDaoImpl类，实现UserDao接口

```
 @Repository("userDao")//此处标注为bean
public class UserDaoImpl implements UserDao {

    @Override
    public void save() {

        System.out.println("userDao...save...");
    }

}
```

UserService接口

```
public interface UserService {
    public void save();

}
```

UserServiceImpl类，实现UserService接口

```

@Service("userService")//标注为bean
public class UserServiceImpl implements UserService {
    @Resource(name="userDao")//按bean的名称装配
    private UserDao userDao;
    public void setUserDao(UserDao userDao){
        this.userDao=userDao;
    }
    @Override
    public void save() {
        // TODO Auto-generated method stub
        this.userDao.save();
        System.out.println("userservice...save...");

    }

}
```

UserController类

B战搜：【拯救者】SSM期末速成

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```java
@Controller("userAction")//标注为bean
public class UserController {
    @Resource(name="userService")//按bean的名称装配
    private UserService userService;
    public void setUserService(UserService userService){
        this.userService=userService;
    }
    public void save(){
        this.userService.save();
        System.out.println("userAction...save...");
    }

}
```

xml的配置

```
  xmlns:context="http://www.springframework.org/schema/context
  http://www.springframework.org/schema/context
  http://www.springframework.org/schema/context/spring-context.xsd

    <!-- 通知spring扫描指定路径，进行注解的解析-->
 <context:component-scan base-package="包名"></context:component-scan>
```

【拯救者】Ep07_Spring的Bean-习题

B战搜:【拯救者】SSM期末速成

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

所谓自动装配，就是将一个 Bean 自动的注入到到其他 Bean 的【 】中。

【 】注解用于将数据访问层（ DAO 层）的类标识为 Spring 中的 Bean，其功能与 @Component 相同。

对于【 】作用域的 Bean，Spring 只负责创建，当容器创建了 Bean 实例后， Bean 的实例就交给客户端代码来管理， Spring 容器将不再跟踪其生命周期。

Spring 的<bean>元素中的 autowire 属性取值不包括以下。 （ ）

A: default B: byName C: byType D: byId

以下有关 Bean 的装配方式说法正确的是。 （ ）

A: Spring 容器支持多种形式的 Bean 的装配方式，如基于 XML 的装配、基于注解（Annotation）的装配和自动装配（其中最常用的是基于 XML 的装配）；

B: Spring 提供了 3 种基于 XML 的装配方式：设值注入、构造注入和属性注入；

C: 在 Spring 实例化 Bean 的过程中，Spring 首先会调用 Bean 的默认构造方法来实例化 Bean 对象，然后通过反射的方式调用 setter 方法来注入属性值；

D: 设值注入要求一个 Bean 必须提供一个有参构造方法并且为需要注入的属性提供对应的 setter 方法。

以下哪些不属于 Spring 4.3 版本中 Bean 的作用域。 （ ）

A: application B: request C: response D: globalSession

Spring 中定义了一系列的注解，以下有关其常用的注解说明错误的是。 （ ）

A: @Autowired 用于对 Bean 的属性变量、属性的 setter 方法及构造方法进行标注，配合对应的注解处理器完成 Bean 的自动配置工作，默认按照 Bean 的名称进行装配。

B: @Repository 用于将数据访问层（ DAO 层）的类标识为 Spring 中的 Bean。

C: @Service 通常作用在业务层（ Service 层），用于将业务层的类标识为 Spring 中的 Bean。

D: @Controller 通常作用在控制层（如 Spring MVC 的 Controller），用于将控制层的类标识为 Spring 中的 Bean。

B战搜：【拯救者】SSM期末速成

提示：

Spring容器支持多种形式的Bean装配方式，如基于XML的装配、基于Annotation（注解）的装配和自动装配等。自动装配(最优选择)

提供了两种基于XML的装配方式：设值注入（Setter Injection）和构造注入（Constructor

请简述基于 XML 的装配方式中设值注入的要求。

1、Bean类必须提供一个默认的无参构造方法。

2、Bean类必须为需要注入的属性提供对应的setter 方法。

【拯救者】Ep08_Spring的AOP(上)

我嚓 OP!

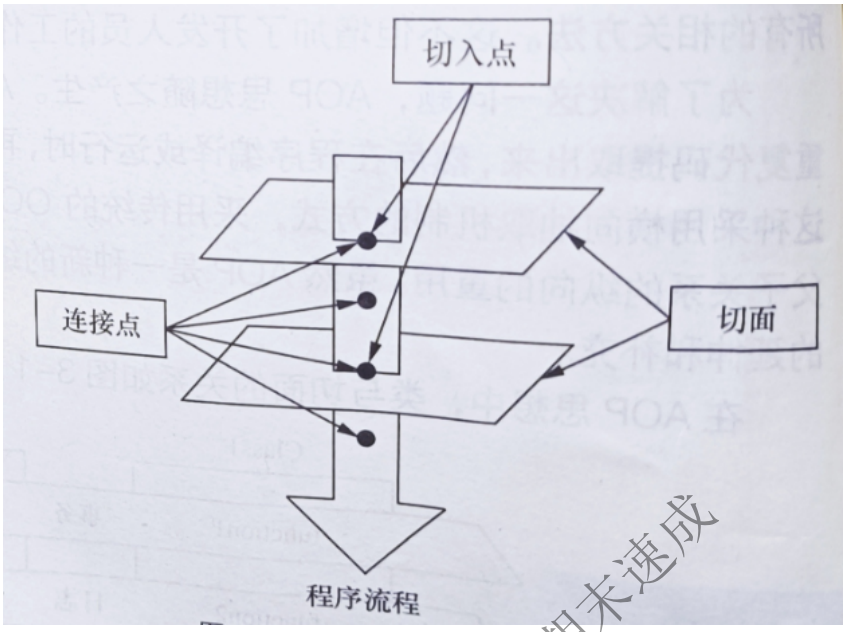
B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

AOP的简介

（Aspect Oriented Programming）中文翻译过来是 面向切面编程
底层实现也是基于 动态代理 完成的。

AOP的相关术语



连接点（JointPoint）: **有可能调用AOP的地方叫做 连接点**

切点（PointCut）: **提供一组规则 来匹配 Join Point**

通知（Advice）: **切面的工作被称之为通知。**

切面（ASPECT）: **由切点（Pointcut）和通知（Advice）组成**

B战搜:【拯救者】SSM期末速成

AOP的实现方式

1.**添加** Spring AOP **框架**支持
2.定义切面和切点
3.**定义通知**

使用AOP织入，需要导入一个依赖包

```
<dependency>
     <groupId>org.aspectj</groupId>
     <artifactId>aspectjweaver</artifactId>
     <version>1.9.4</version>
 </dependency>
```

UserService接口

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
public interface UserService {
    public void add();
    public void delete();
    public void update();
    public void select();
}
```

UserServiceImpl实现类

```
public class UserServiceImpl implements UserService{
    @Override
    public void add() {
        System.out.println("增加了一个用户");
    }

    @Override
    public void delete() {
        System.out.println("删除了一个用户");

    }

    @Override
    public void update() {
        System.out.println("更新了一个用户");

    }

    @Override
    public void select() {
        System.out.println("查询了一个用户");

    }
}
```

B战搜:【拯救者】SSM期末速成

自定义类

```
public class MyPointCut {
    public void before(){
        System.out.println("==========方法执行前 预处理==========");
    }

    public void after(){
        System.out.println("==========方法执行后 执行完毕==========");
    }
}
```

xml

```
       xmlns:aop="http://www.springframework.org/schema/aop
        http://www.springframework.org/schema/aop
```

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
        http://www.springframework.org/schema/aop/spring-aop.xsd

        <!--注册bean-->
    <bean id="userService" class="com.wuji.service.impl.UserServiceImpl"/>
    <bean id="myPt" class="com.wuji.MyPointCut"/>
    <aop:config>

        <!--自定义切面，ref 要引用的类-->
        <aop:aspect ref="myPt">

            <!--切入点-->
            <aop:pointcut id="point" expression=    "execution(
            void com.wuji.service.impl.UserServiceImpl.select()) "/>

            <!--通知-->
            <aop:before method="before" pointcut-ref="point"/>
            <aop:after  method="after" pointcut-ref="point"/>

        </aop:aspect>

    </aop:config>
    </beans>
```

测试类

```
public class MyTest {
    public static void main(String[]  args) {
        ApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml");
        //动态代理的是接口
        UserService userService = (UserService) context.getBean("userService");
        userService.select();

    }
}
```

B战搜:【拯救者】SSM期末速成

基于注解的AOP编程

切面类

```
@Aspect //标注这个类是一个切面
public class AnnotationPointCut {
   @Before("execution(void service.UserServiceImpl.select( ))")
   public void before(){
       System.out.println("============方法执行前============");
   }

   @After("execution(void service.UserServiceImpl.select( ))")
   public void after(){
       System.out.println("============方法执行后============");
   }
}
```

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

applicationContext.xml

```
      xmlns:aop="http://www.springframework.org/schema/aop
      http://www.springframework.org/schema/aop
      http://www.springframework.org/schema/aop/spring-aop.xsd

        <!--注册bean-->
    <bean id="userService" class="com.wuji.service.impl.UserServiceImpl"/>
    <bean id="myPt" class="com.wuji.MyPointCut"/>
    <bean id="annotationPointCut" class="com.wuji.AnnotationPointCut"/>

        <!--开启注解支持-->
     <aop:aspectj-autoproxy proxy-target-class="true"/>
```

【拯救者】Ep09_Spring的AOP(下)

通知类型

```
1 前置通知使用 @Before：通知方法会在目标方法调用之前执行。
2 后置通知使用 @After：通知方法会在目标方法返回或者抛出异常后调用。
3 返回之后通知使用 @AfterReturning：通知方法会在目标方法返回后调用。
4 抛异常后通知使用 @AfterThrowing：通知方法会在目标方法抛出异常后调用。
5 环绕通知使用 @Around：通知包裹了被通知的方法，在被通知的方法通知之前和调用之后执行自定义的行为。
   public void around(ProceedingJoinPoint proceedingJoinPoint) throws Throwable
{
        System.out.println("环绕前");
          //执行方法
        Object proceed = proceedingJoinPoint.proceed();
        System.out.println("环绕后");

   }
```

切点表达式

```
execution() 是最常用的切点函数，用来匹配方法，语法为：
    execution(<修饰符>(一般省略)  <返回类型>    <包.类.方法(参数)>  )
         修饰符：    public, private 等，省略时匹配任意修饰符
         返回类型：   * 匹配任意类型
         匹配包：     .. 匹配包及其子包的所有类
         目标类：     省略时匹配任意类型 , + :  必须跟在类名后面，如 com.Car+ ,表示继承该类的子类包括本身
         方法：       * 匹配任意方法 ,  user* 匹配名称以 user 开头的方法
         参数:        () 匹配没有参数的方法  (..) 匹配有任意数量参数的方法 (*) 匹配有一个任意类型参数的方法
                      (*,String) 匹配有两个参数的方法，并且第一个为任意类型，第二个为 String 类型
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
execution(* com.wuji.User.*(..)) ：匹配 User 类里的所有方法。
execution(* com.wuji.User+.*(..)) ：匹配该类的子类包括该类的所有方法。
execution(* com.wuji.*.*(..)) ：匹配 com.wuji 包下的所有类的所有方法。
execution(* com..*.(..))`:匹配任意返回类型，com包、子包下所有类的所有方法。
```

【拯救者】Ep10_Spring的AOP-习题

在Spring配置文件中，< aop:aspect >子元素的（）属性用于指定一个已经存在的切入点名称。

AOP采取（）抽取机制，将分散在各个方法中的重复代码提取出来，然后在程序编译或运行时，再将这些提取出来的代码应用到需要执行的地方。

目前最流行的AOP框架有两个，分别为SpringAOP和（）。

AspectJ 框架中注解【　　】用于定义切入点表达式，在使用时还需定义一个包含名字和任意参数的方法签名来表示切入点名称。

在 Spring 配置文件中， <aop:aspect>子元素的 pointcut-ref 属性用于指定一个已经存在的【　　】。

在 Spring 的配置文件中，配置切面使用的是【　　】元素。

AOP 术语中【　　】表示 AOP 框架在特定的切入点执行的增强处理，即在定义好的切入点处所要执行的程序代码。

以下哪种类型不是 Spring 中的通知类型。（　）

A: 异常通知

B: 前置通知

C: 后置通知

D: 最终通知

以下关于Spring AOP 的介绍错误的是（）。

、AOP的全称是Aspect-Oriented Programming，即面向切面编程（也称面向方面编程）
、AOP采取横向抽取机制，将分散在各个方法中的重复代码提取出来，这种采用横向抽取机制的方式，采用OOP思想是无法办到的
、虽然AOP是一种新的编程思想，采取横向抽取机制，是OOP的升级替代品
、目前最流行的AOP框架有两个，分别为Spring AOP和AspectJ

以下不属于ProxyFactoryBean类中的常用可配置属性的是（）。

A、target
B、proxyInterfaces
C、targetClass
D、interceptorNames

请对 Spring 的通知类型进行简要说明。

B战搜：【拯救者】SSM期末速成

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

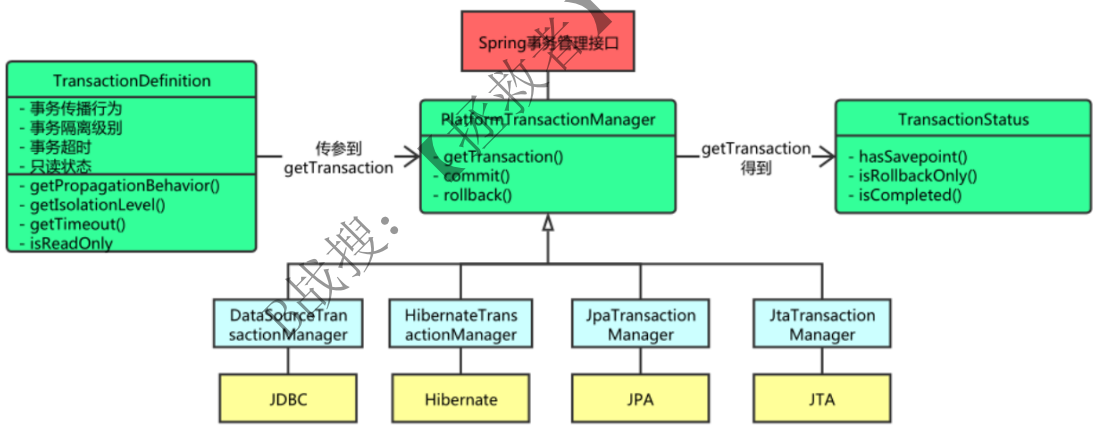
属性名称	描述
target	代理的目标对象
proxyInterfaces	代理要实现的接口，如果是多个接口，可以使用以 <list> <value></value> ... </list>
proxyTargetClass	是否对类代理而不是接口，设置为 true 时，使用 C
interceptorNames	需要织入目标的 Advice
singleton	返回的代理是否为单实例，默认为 true（即返回单
optimize	当设置为 true 时，强制使用 CGLIB

【拯救者】Ep11_Spring事务管理(上)

事务的概念

就是为了达到某个目的，而做的一系列的操作，要么一起成功(事务提交)，要么一起失败(事务回滚)。

Spring事务管理



Spring事务管理器的接口是 **PlatformTransactionManager**，通过这个接口，Spring为各个平台如JDBC、Hibernate等都提供了对应的事务管理器

```
Public interface PlatformTransactionManager()...{
        // 由TransactionDefinition得到TransactionStatus对象
        TransactionStatus getTransaction(TransactionDefinition definition)
throws TransactionException;
        // 提交
        Void commit(TransactionStatus status) throws TransactionException;
        // 回滚
        Void rollback(TransactionStatus status) throws TransactionException;
 }
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

JDBC事务管理

JDBC（Java Database Connectivity）是 Java 中用于与数据库进行交互的 API。JDBC 提供了一种在 Java 程序中管理 数据库事务 的方式。

如果应用程序中直接使用JDBC来进行持久化，DataSourceTransactionManager会为你处理事务边界

```
    <bean id="transactionManager"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
        <property name="dataSource" ref="dataSource" />
    </bean>
```

TransactionDefinition类

这个类就定义了一些**基本的事务属性。**

```
public interface TransactionDefinition {
    int getPropagationBehavior(); // 返回事务的传播行为
    int getIsolationLevel(); // 返回事务的隔离级别，事务管理器根据它来控制另外一个事务可以
看到本事务内的哪些数据
    int getTimeout();  // 返回事务必须在多少秒内完成
    boolean isReadOnly(); // 事务是否只读，事务管理器能够根据这个返回值进行优化，确保事务是
只读的
}
```

Spring提供了对编程式事务和声明式事务的支持

编程式事务 侵入到了业务代码里面，但是提供了更加详细的事务管理

声明式事务由于基于AOP，所以既能起到事务管理的作用，又可以**不影响业务代码的具体实现。**

编程式事务管理

```
DataSourceTransactionManager dataSourceTransactionManager = new
DataSourceTransactionManager(); //定义一个某个框架平台的TransactionManager，如JDBC、
Hibernate

 dataSourceTransactionManager.setDataSource(this.getJdbcTemplate().getDataSource
()); // 设置数据源
    DefaultTransactionDefinition transDef = new DefaultTransactionDefinition();
// 定义事务属性

 transDef.setPropagationBehavior(DefaultTransactionDefinition.PROPAGATION_REQUIR
ED); // 设置传播行为属性
    TransactionStatus status =
dataSourceTransactionManager.getTransaction(transDef); // 获得事务状态
    try {
        // 数据库操作
        dataSourceTransactionManager.commit(status);// 提交
    } catch (Exception e) {
        dataSourceTransactionManager.rollback(status);// 回滚
    }
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

声明式事务管理

```
    <!-- 数据库驱动 和 连接池-->
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>8.0.25</version>
    </dependency>

    <dependency>
      <groupId>com.alibaba</groupId>
      <artifactId>druid</artifactId>
      <version>1.2.8</version>
    </dependency>

    <!-- spring-jdbc -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-jdbc</artifactId>
      <version>5.3.28</version>
    </dependency>

    <!-- 声明式事务依赖-->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-tx</artifactId>
      <version>5.3.28</version>
    </dependency>
    <dependency>
      <groupId>com.mchange</groupId>
      <artifactId>c3p0</artifactId>
      <version>0.9.5</version>
    </dependency>
```

使用tx标签配置的拦截器

```
 xmlns:tx="http://www.springframework.org/schema/tx
 http://www.springframework.org/schema/tx
 http://www.springframework.org/schema/tx/spring-tx-2.5.xsd

     <context:annotation-config />
    <context:component-scan base-package="com.wuji" />

   <!-- 生成连接池 -->
  <bean class="com.mchange.v2.c3p0.ComboPooledDataSource" id="dataSource">
    <property name="driverClass" value="com.mysql.jdbc.Driver"></property>
    <property name="jdbcUrl" value="jdbc:mysql://localhost:3306/ssmtest">
</property>
    <property name="user" value="root"></property>
    <property name="password" value="123456"></property>
    <property name="minPoolSize" value="5"></property>
    <property name="maxPoolSize" value="20"></property>
    <property name="initialPoolSize" value="5"></property>
  </bean>
```

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
  <bean class="org.springframework.jdbc.core.JdbcTemplate" id="j">
    <property name="dataSource" ref="dataSource"></property>
  </bean>
  <!-- 配置实体类 -->
  <bean class="com.wuji.dao.impl.UserDaoImpl" id="userDao">
    <property name="jdbcTemplate" ref="j"></property>
  </bean>
  <bean class="com.wuji.service.impl.UserServiceImpl" id="userService">
    <property name="userDAO" ref="userDao"></property>
  </bean>


  <!-- 配置事务管理器 -->
  <bean class="org.springframework.jdbc.datasource.DataSourceTransactionManager"
id="transactionManager">
    <property name="dataSource" ref="dataSource"></property>
  </bean>
  <tx:advice id="advice"     transaction-manager="transactionManager">
    <tx:attributes>
      <tx:method name="*" propagation="REQUIRED"/><!-- *是对所有方法都加 -->
    </tx:attributes>
  </tx:advice>
  <!-- 用切点把事务切进去 -->
  <aop:config>
    <aop:pointcut expression="execution(* com.wuji.*.*.*(..))" id="pointcut"/>
    <aop:advisor advice-ref="advice" pointcut-ref="pointcut"/>
  </aop:config>
```

B战搜:【拯救者】SSM期末速成

【拯救者】Ep12_Spring事务管理(下)

注解实现

```
SprignConfig 加上@EnableTransactionManagement
```

Spring事务的传播方式和隔离级别

传播方式

```
1.propagation_required: 默认事务类型，如果没有，就新建一个事务；如果有，就加入当前事务。适合绝大多数情况。

2.propagation_required_new: 如果没有，就新建一个事务；如果有，就将当前事务挂起。

3.propagation_nested: 如果没有，就新建一个事务；如果有，就在当前事务中嵌套其他事务。

4.propagation_supports: 如果没有，就以非事务方式执行；如果有，就使用当前事务。

5.propagation_not_supported: 如果没有，就以非事务方式执行；如果有，就将当前事务挂起。即无论如何不支持事务。

6.propagation_never: 如果没有，就以非事务方式执行；如果有，就抛出异常。
```

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
7.propagation_mandatory: 如果没有，就抛出异常；如果有，就使用当前事务。
```

讲log案例

隔离级别

隔离级别	含义
ISOLATION_DEFAULT	使用后端数据库默认的隔离级别
ISOLATION_READ_UNCOMMITTED	最低的隔离级别，允许读取尚未提交的数据变更，可能会导致脏读、幻读或不可重复读
ISOLATION_READ_COMMITTED	允许读取并发事务已经提交的数据，可以阻止脏读，但是幻读或不可重复读仍有可能发生
ISOLATION_REPEATABLE_READ	对同一字段的多次读取结果都是一致的，除非数据是被本身事务自己所修改，可以阻止脏读和不可重复读，但幻读仍有可能发生
ISOLATION_SERIALIZABLE	最高的隔离级别，完全服从ACID的隔离级别，确保阻止脏读、不可重复读以及幻读，也是最慢的事务隔离级别，因为它通常是通过完全锁定事务相关的数据库表来实现的

【拯救者】Ep13_Spring事务管理-习题

B战搜:【拯救者】SSM期末速成

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

使用基于注解方式的事务管理需要在使用事务的 Spring Bean 类或者 Bean 类的方法上添加注解【 】。

Spring 的声明式事务管理可以通过两种方式来实现，一种是基于 XML 的方式，另一种是基于【 】的方式。

Spring2.0 以后，提供了 tx 命名空间来配置事务， tx 命名空间下提供了【 】元素来配置事务的通知（增强处理） 。

声明式事务管理：是通过【 】实现的事务管理。

以下关于 @Transactional 注解可配置的参数信息及秒正确的是。 （ ）

A: value 用于指定需要使用的事务管理器，默认为 "" ；

B: read-only 用于指定事务是否只读，默认为 true；

C: isolation 用于指定事务的隔离级别，默认为 Isolation.READ_COMMITTED

D: propagation 用于指定事务的传播行为，默认为 Propagation. SUPPORTS；

以下基于 XML 方式的声明式事务管理配置文件中 <tx:advice> 元素的子元素 <tx:method> 属性描述错误的是。 （ ）

A: name：该属性为必选属性，它指定了与事务属性相关的方法名

B: propagation ：用于指定事务的传播行为，它的默认值为 SUPPORTS

C: isolation ：该属性用于指定事务的隔离级别，其默认值为 DEFAULT

D: read-only：该属性用于指定事务是否只读，其默认值为 false

以下有关事务管理方式相关说法错误的是。 （ ）

A: Spring 中的事务管理分为两种方式：一种是传统的编程式事务管理，另一种是声明式事务管理。

B: 编程式事务管理： 是通过 AOP 技术实现的事务管理， 就是通过编写代码实现的事务管理包括定义事务的开始、正常执行后的事务提交和异常时的事务回滚。

C: 声明式事务管理： 其主要思想是将事务管理作为一个 “切面 ”代码单独编写， 然后通过 AOP 技术将事务管理的 “切面 ”代码植入到业务目标类中。

D: 声明式事务管理最大的优点在于开发者无需通过编程的方式来管理事务，只需在配置文件中进行相关的事务规则声明，就可以将事务规则应用到业务逻辑中。

以下有关 Spring 事务管理及核心接口说法错误的是。 （ ）

A: PlatformTransactionManager 接口是 Spring 提供的平台事务管理器，主要用于管理事务。

B: TransactionDefinition 接口是事务定义（描述）的对象，该对象中定义了事务规则，并提供了获取事务相关信息的方法。

C: TransactionStatus 接口是事务的状态，它描述了某一时间点上事务的状态信息。

D: 在事务管理过程中，传播行为可以控制是否需要创建事务以及如何创建事务，通常情况下，对于数据的查询、插入、更新和删除操作，必须进行事务管理。

下列选项中，哪一个不是 Spring 中事务管理的核心接口。 （ ）

A: PlatformTransactionManager

B: TransactionDefinition

C: TransactionStatus

D: TransactionManager

B战搜：【拯救者】SSM期末速成

【拯救者】Ep14_MyBatis(上)

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

MyBatis简介

MyBatis 是一款优秀的持久层框架
方便程序员将数据存入到数据库中。
传统的JDBC代码太复杂了。避免了几乎所有的 JDBC 代码和手动设置参数以及获取结果集。。

使用步骤:

1.引入依赖

```
<dependency>
    <groupId>org.mybatis</groupId>
    <artifactId>mybatis</artifactId>
    <version>3.5.3</version>
</dependency>
```

2.新建Dao接口

```
public interface UserDao {
    List<User> getUserList();
}
```

3.UserDaoMapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
        PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<!--namespace绑定一个对应的Mapper接口-->
<mapper namespace="com.newer.dao.UserDao">
   <select id="getUserList" resultType="com.wuji.entity.User">
       select * from user
   </select>
</mapper>
```

4.编写mybatis的核心配置文件

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
        PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-config.dtd">
<!--configuration核心配置文件-->
<configuration>
    <!--environments配置环境组-->
    <!--default默认环境-->
    <environments default="development">
        <!--environment单个环境-->
        <environment id="development">
            <!--transactionManager配置事务管理器-->
            <transactionManager type="JDBC"/>
            <!--配置连接池-->
```

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
            <dataSource type="POOLED">
                <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
                <property name="url" value="jdbc:mysql://localhost:3306/ssmtest?
serverTimezone=UTC"/>
                <property name="username" value="root"/>
                <property name="password" value="123456"/>
            </dataSource>
        </environment>
    </environments>

    <!--每一个Mapper.xml需要在Mybatis核心配置文件中注册-->
    <mappers>
        <mapper resource="com/wuji/dao/xml/UserMapper.xml"/>
    </mappers>
</configuration>
```

测试

```
     InputStream inputStream = Resources.getResourceAsStream("mybatis-
config.xml");
        SqlSessionFactory sqlSessionFactory = new
SqlSessionFactoryBuilder().build(inputStream);

        //2.获取SqlSession对象，用它来执行sql
        SqlSession sqlSession = sqlSessionFactory.openSession();

        //3.执行sql
        List<User> users = sqlSession.selectList("namespace+id");

        System.out.println(users);

        //4.释放资源
        sqlSession.close();
```

下面的要提交事务

基于MyBatis的添加

```
<insert id="add" parameterType="com.wuji.entity.User">
    insert into user values(#{id},#{username},#{money})
</insert>
```

基于MyBatis的删除

```
<delete id="delete" parameterType="java.lang.Integer">
    delete from user where id=#{id}
</delete>
```

基于MyBatis的修改

```
<update id="add" parameterType="com.wuji.entity.User">
    update user set username=#{username} , money=#{money} where id=#{id}
</update>
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

语法#和$

```
1.#{}是占位符：动态解析->预编译->执行  Mybatis会将sql中的#{}替换为?号，在sql执行前会使用PrepareStatement的参数设置方法，按序给sql的?号占位符设置参数值，能防止sql注入
2.${}是拼接符：动态解析->编译->执行 ,${}会将传递来的参数与其他sql部分进行拼接成一个完整sql语句（类似于字符串拼接,不能防止sql注入
```

【拯救者】Ep15_MyBatis(下)

基于MyBatis的查询

单条记录查询

resultType=直接写类就行

多条记录查询

1.list ----->resultType=直接写类就行

2.map ----->resultType="map"

```
Map < String , Object > getAllUserToMap ();
```

MyBatis的结果类型

resultType

resultType 可以把查询结果封装到 pojo 类中，但必须 pojo类 的**属性名**和查询到的**数据库表的字段名**一致

resultMap

sql列名和pojo类 的属性名封装类对象属性名**不同**的，则需要使用**resultMap**将字段名和属性名对应起来，进行封装，将结果映射到pojo类对象中。

```
<resultMap type="order" id="orderResultMap">
     <!-- property：主键在pojo中的属性名 -->
     <!-- column：主键在数据库中的列名 -->
     <result column="brand_name" property="brandName"/>
     <result column="company_name" property="companyName"/>
 </resultMap>
```

【拯救者】Ep16_MyBatis-习题

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

MyBatis 的删除操作在映射文件中是通过配置【 】元素来实现的。

MyBatis 映射文件中 <mapper> 元素是配置文件的根元素，它包含一个【 】属性，该属性为这个 <mapper> 指定了唯一的命名空间。

【 】是当前主流的 Java 持久层框架之一， 它与 Hibernate 一样，也是一种 ORM 框架。

MyBatis 的删除操作有关说发错误的是。 （ ）

A: MyBatis 的删除操作在映射文件中是通过配置 <delete> 元素来实现的。

B: MyBatis 的删除操作也需要进行事务提交。

C: MyBatis 的删除操作执行了 SqlSession 的 delete()方法。

D: MyBatis 的删除操作和添加操作时，也需要封装整个实体类。

MyBatis 的更新操作有关说发错误的是。 （ ）

A: MyBatis 的更新操作在映射文件中是通过配置 <update> 元素来实现的 。

B: MyBatis 的更新操作也需要进行事务提交。

C: MyBatis 的更新操作执行了 SqlSession 的 update()方法 。

D: MyBatis 的更新操作和添加操作时，只需要将 insert()方法改为 updat()方法即可。

关于 MyBatis 模糊查询中进行 SQL 字符串拼接时，说法错误的是。 （ ）

A: 使用 “ ${}进行 SQL 字符串拼接时，无法防止 SQL 注入问题。

B: 可以使用 MySQL 中的 concat()函数进行字符串拼接。

C: 使用 MySQL 中的 concat()函数进行字符串拼接，也无法防止 SQL 注入。

D: 使用 MySQL 中的 concat()函数进行字符串拼接，导致数据库移植性变差。

B站搜：【拯救者】SSM期末速成

【拯救者】Ep17_MyBatis的动态SQL语句(上)

前面我们的 SQL 都是比较简单的，有些时候业务逻辑复杂时，**我们的 SQL 是动态变化的**，此时在前面的学习中我们的 SQL 就不能满足要求了。

if

```
<select id="findByCondition" resultType="User" parameterType="User">
        select * from user where  1=1
        <if test="username !=null and username !=''">
            and username = #{username}
        </if>
        <if test="address !=null">
            and address = #{address};
        </if>
</select>
```

where

为了简化上面 `where 1=1` 的条件拼装，我们可以采用`where`标签来简化开发

B站搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
<select id="findByCondition" resultType="User" parameterType="User">
        select * from user
        <where>
            <if test="username !=null and username !=''">
                and username  = #{username}
            </if>
            <if test="address !=null">
                and address  = #{address};
            </if>
        </where>
    </select>
```

set（主要用于更新时）

它的主要功能和where元素其实是差不多的，如果包含的语句是**以逗号结束的话将会把该逗号忽略**

```
  <update id="dynamicSetTest" parameterType="Article">
        update article
        <set>
            <if test="title != null">
                title = #{title},
            </if>
            <if test="content != null">
                content = #{content}
            </if>
        </set>
        where id = #{id}
    </update>
```

B战搜：【拯救者】SSM期末速成

trim

`prefix|suffix`：将 trim 标签中内容前面或后面添加指定内容

`suffixOverrides|prefixOverrides`：将trim标签中内容前面或后面去掉指定内容

```
    <trim prefix="(" prefixOverrides="OR" suffixOverrides="," suffix=")">子句
</trim>
      作用就是：去掉子句首的OR和子句尾的逗号，并在子句前后分别加上(和)，比如，"orabc,"-->"
(abc)"。

    WHERE a = #{a}
    <trim prefix="AND(" prefixOverrides="OR" suffix=")">
        <if test="b != -1">
            OR b = #{b}
        </if>
        <if test="c != -1">
            OR c = #{c}
        </if>
    </trim>
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

【拯救者】Ep18_MyBatis的动态SQL语句(下)

foreach

```
<foreach>标签用于遍历集合，它的属性：
    collection:代表要遍历的集合元素，注意编写时不要写#{}
    open:代表语句的开始部分
    close:代表结束部分
    item:代表遍历集合的每个元素，生成的变量名,需要和#{item}名称对应
    sperator:代表分隔符
```

SELECT * FROM user WHERE id IN (1,2,3)

```
 <select id="findUserInIds" resultType="User" parameterType="list">
             SELECT * FROM user
        <where>
                <foreach collection="list" open="id in (" close=")" item="id"
separator=",">
                    #{id}
                </foreach>
        </where>
    </select>
```

choose、when、otherwise 标签

```
choose、when、otherwise，相当于if...else if...else
     <select id="getStuByChoose" resultType="user">
        select * from stu
        <where>
            <choose>
                <when test="sid != null and sid != ''">
                    sid = #{sid}
                </when>
                <when test="sname != null and sname != ''">
                    sname = #{sname}
                </when>
                <when test="score != null and score != ''">
                    score = #{score}
                </when>
                <otherwise>
                    sid = 5
                </otherwise>
            </choose>
        </where>
    </select>
```

B战搜：【拯救者】SSM期末速成

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

SQL抽取

```
<!-- 抽取重复的语句代码片段 -->
<sql id="defaultSql">
    select * from user
</sql>

<include refid="defaultSql"> </include>
```

【拯救者】Ep19_MyBatis的动态SQL语句-习题

B战搜:【拯救者】SSM期末速成

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

在 MyBaits 映射文件中使用 <set>和<if> 元素组合进行 update语句动态 SQL 组装时，如果<set>元素内包含的内容都为空，则会出现【 】。

MyBaits 中<trim> 元素的作用是去除一些特殊的字符串，它的【 】属性代表的是语句的前缀，而 prefixOverrides 属性代表的是需要去除的哪些特殊字符串。

使用 <set>和<if> 元素相结合的方式来组装 update 语句时，<set>元素就会动态前置 【 】关键字，同时也会消除 SQL 语句中最后一个多余的【 】。

以下不属于 <foreach> 元素中使用的属性的是。 （ ）

A: separator

B: collection

C: current

D: item

以下关于 <foreach> 元素中使用的几种属性的描述错误的是。 （ ）

A: item ：配置的是循环中当前的元素。

B: index ：配置的是当前元素在集合的位置下标。

C: collection ：配置的是传递过来的参数类型， 它可以是一个 array、list（或 collection）、Map 集合的键、 POJO 包装类中数组或集合类型的属性名等。

D: separator：配置的是各个元素的间隔符。

以下关于 MyBatis 的<set>元素的使用及说法正确的是。 （ ）

A: <set> 元素主要用于更新操作，其主要作用是在动态包含的 SQL 语句前输出一个 SET 关键字，并将 SQL 语句中最后一个多余的逗号去除。

B: 使用 MyBatis 的<set>元素来更新操作时，前端需要传入所有参数字段，否则未传入字段会默认设置为空。

C: 在映射文件中使用 <set>和<if> 元素组合进行 update 语句动态 SQL 组装时， <set>元素内包含的内容可以都为空， <if> 元素会进行判断处理。

D: 在映射文件进行更新操作时，只需要使用 <set>元素就可以进行动态 SQL 组装。

以下有关 MyBatis 动态 SQL 中的主要元素说法错误的是。 （ ）

A: <if> 用于单条件分支判断。

B: <choose> （ <when>、 <otherwise> ）用于多条件分支判断。

C: <foreach> 循环语句，常用于 in 语句等列举条件中。

D: <bind> 从 OGNL 表达式中创建一个变量， 并将其绑定到上下文， 只于模糊查询的 sql 中。

当有多个选择的情况是，值需要从多个选项中选择一个去执行时，可以使用的动态 SQL 元素是。（ ）

A: <if>

B: <choose> 、 <when>、 <otherwise>

C: <when>

D: <set>

B战搜：【拯救者】SSM期末速成

补充:

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

bind 标签可以通过 OGNL 表达式自定义一个上下文变量。

比如，按照网站名称进行模糊查询，SQL 映射文件如下。

```
<select id="selectWebsite" resultType="net.biancheng.po.Website">
    <bind name="pattern" value="'%'+_parameter+'%'" />
    SELECT id,name,url,age,country
    FROM website
    WHERE name like #{pattern}
</select>
```

bind 元素属性如下。

- value：对应传入实体类的某个字段，可以进行字符串拼接等特殊处理。
- name：给对应参数取的别名。

以上代码中的“_parameter”代表传递进来的参数，它和通配符连接后，赋给了 pattern，然后就可以在 select 语句中使用这个变量进行模糊查询，不管是 MySQL 数据库还是 Oracle 数据库都可以使用这样的语句，提高了可移植性。

【拯救者】Ep20_SpringMVC基础(上)

SpringMVC概述

一种轻量级的、基于MVC的Web层应用框架

MVC，

M：model，模型层，指的是项目中的实体JavaBean，装载并传输数据；

V：View 视图层， 它可以是 JSP、HTML、FreeMarker、Thymeleaf

C：Controller 控制层，指的是项目中的Servlet，根据请求的 URL 映射到不同的控制器方法，然后将模型数据传递给视图进行渲染。

入门案例

导入包

```
<dependencies>
   <!--spring mvc-->
   <dependency>
     <groupId>org.springframework</groupId>
     <artifactId>spring-webmvc</artifactId>
     <version>5.3.25</version>
   </dependency>
   <!--servlet-->
   <dependency>
     <groupId>javax.servlet</groupId>
     <artifactId>javax.servlet-api</artifactId>
     <version>3.1.0</version>
     <!--provided仅在编译期间使用，项目打包不包含这个依赖，并且这个依赖不会被传递-->
     <scope>provided</scope>
   </dependency>
 </dependencies>
```

> resources下创建spring-mvc.xml文件。

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
        xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
https://www.springframework.org/schema/context/spring-context.xsd
http://www.springframework.org/schema/mvc
https://www.springframework.org/schema/mvc/spring-mvc.xsd">
    <!--开启注解-->
    <context:component-scan base-package="com.wuji"/>
    <!--启用mvc（适配器、映射器）-->
    <mvc:annotation-driven></mvc:annotation-driven>
    <!--视图解析器-->
     <bean id="internalResourceViewResolver"
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <!-- 设置视图前缀路径 -->
        <property name="prefix" value="WEB-INF/page/"/>
        <!--  设置视图指定文件的后缀 -->
        <property name="suffix" value=".jsp"/>
    </bean>
</beans>
```

web.xml中配置DispatcherServlet。

```
<!DOCTYPE web-app PUBLIC
 "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
 "http://java.sun.com/dtd/web-app_2_3.dtd" >
<web-app>
  <!--配置前端控制器DispatcherServlet-->
  <servlet>
    <servlet-name>dispatcherServlet</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-
class>
    <!--加载Spring MVC配置文件-->
    <init-param>
      <param-name>contextConfigLocation</param-name>
      <param-value>classpath:spring-mvc.xml</param-value>
    </init-param>
    <!--启动容器时候加载servlet-->
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>dispatcherServlet</servlet-name>
    <!--表示拦截所有请求-->
    <url-pattern>/</url-pattern>
  </servlet-mapping>
</web-app>
```

hello.jsp

B战搜：【拯救者】SSM期末速成

B战搜：【拯救者】SSM期末速成

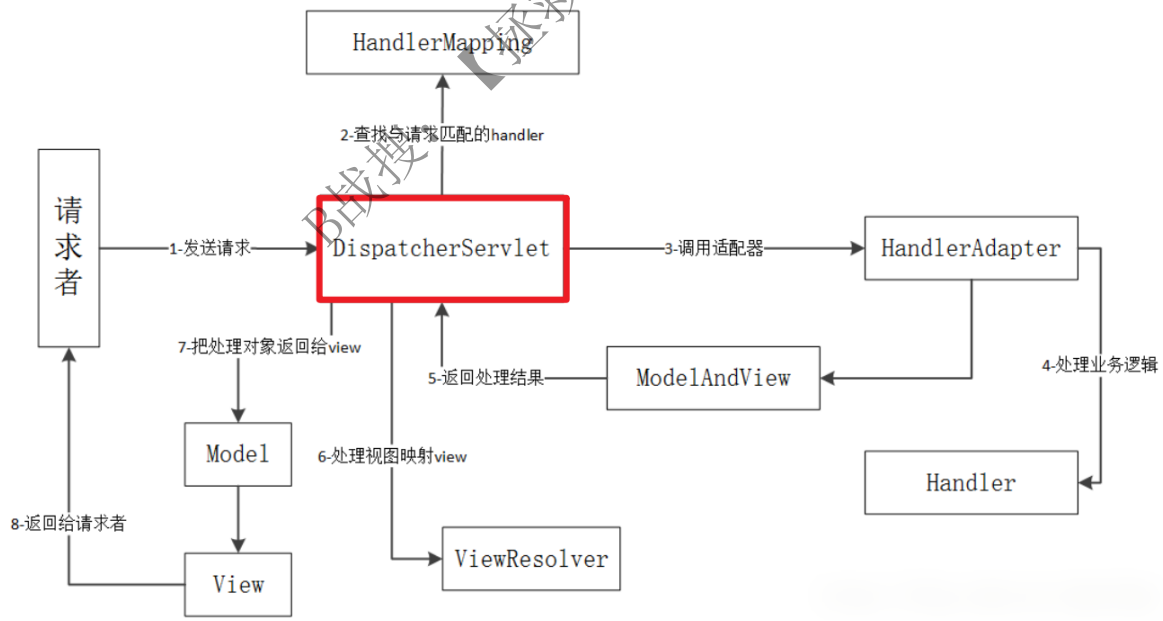
定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```jsp
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<html>
    <head>
        <title>hello</title>
    </head>
    <body>
        hello ZWJ hello页面
    </body>
</html>
```

创建UserController类

```java
@Controller
public class HelloController {
    @RequestMapping("/hello")
    public String hello(){
        //跳转到/pages/hello.jsp页面
        return "hello";
    }
    @RequestMapping("/hello2")
    @ResponseBody
    public String hello2(){
        return "helloZWJ";
    }
}
```

HelloWorld的运行流程



B战搜【拯救者】SSM期末速成

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

（1）客户端（浏览器）发送请求，直接请求到DispatcherServlet。
（2）DispatcherServlet根据请求信息调用HandlerMapping，解析请求对应的Handler。
（3）解析到对应的Handler后，开始由HandlerAdapter适配器处理。
（4）HandlerAdapter会根据Handler来调用真正的处理器开处理请求，并处理相应的业务逻辑。
（5）处理器处理完业务后，会返回一个ModelAndView对象，Model是返回的数据对象，View是个逻辑上的View。
（6）ViewResolver会根据逻辑View查找实际的View。
（7）DispaterServlet把返回的Model传给View。
（8）通过View返回给请求者（浏览器）

注解式控制器简介

@Controller：用于标识是处理器类；
@RequestMapping：请求到处理器功能方法的映射规则；
@RequestParam：请求参数到处理器功能处理方法的方法参数上的绑定；
@ModelAttribute：请求参数到命令对象的绑定；
@RequestHeader：请求头（header）数据到处理器功能处理方法的方法参数上的绑定；
@RequestBody：请求的body体的绑定（通过HttpMessageConverter进行类型转换）；
@ResponseBody：处理器功能处理方法的返回值作为响应体（通过HttpMessageConverter进行类型转换）；
@PathVariable：请求URI中的模板变量部分到处理器功能处理方法的方法参数上的绑定，

【拯救者】SSM期末速成
B战搜：

【拯救者】Ep21_SpringMVC基础(下)

REST简介

REST简介（Representational State Transfer），表现形式状态转换

传统风格资源描述形式

```
http://localhost/user/getById?id=1
http://localhost/user/saveUser
```

REST风格描述形式

```
http://localhost/user/1
http://localhost/user
```

@RequestMapping

@RequestMapping 注解为控制器指定可以处理哪些 URL 请求

```
value = "/testMethod", method = RequestMethod.POST
指定请求方式为POST方式，如此便不再响应GET方式的请求。
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

RequestMapping 支持 Ant 风格的 URL。
---?：匹配文件名中的一个字符
---*：匹配文件名中的任意字符
---**：** 匹配多层路径
示例：
    ---/user/*/createUser：可以匹配如下
    /user/aaa/createUser、/user/bbb/createUser 等 URL。
    ---/user/**/createUser：可以匹配如下
    /user/createUser、/user/aaa/bbb/createUser 等 URL。
    ---/user/createUser??：可以匹配如下
    /user/createUseraa、/user/createUserbb 等 URL，但不匹配/user/createUseraaa。

@PathVariable

通过 @PathVariable 可以将 URL 中占**位符参数绑定到控制器处理方法的入参中**：

URL 中的 **{xxx} 占位符可以通过@PathVariable("xxx")** 绑定到操作方法的入参中。

```
    @RequestMapping("/testPathVariable/{id}")
    public String testPathVariable(@PathVariable("id") Integer id) {
                System.out.println("testPathVariable: " + id);
        return "zwj";
    }
```

假如请求为/testPathVariable/1，那么参数"1"会作为参数传给id作为传入参数。在方法中就可以获取这个参数值。

请求处理方法签名

Spring MVC 通过分析处理方法的签名，可以将 HTTP 请求信息绑定到处理方法的相应人参中

@RequestParam

可以把请求参数传递给请求方法。有三个属性：

value：指定参数名
required：说明是否必须。默认为 true，表示请求参数中必须包含对应的参数，若不存在，将抛出异常
defaultValue：指定请求参数的默认值。

```
  @RequestMapping(value = "/testRequestParam")
    public String testRequestParam(
            @RequestParam(value = "username") String un,
            @RequestParam(value = "age", required = false, defaultValue = "10")
int age) {
        System.out.println("testRequestParam, username: " + un + ", age: "
                + age);
        return "zwj";
    }
```

声明两个输入参数，其中age参数可以缺省，默认为10。如/testRequestParam?
username=kang&age=11。

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

@RequestHeader

可以绑定**请求报头的属性值。**

```
    @RequestMapping("/testRequestHeader")
public String testRequestHeader(
        @RequestHeader(value = "Accept-Language") String al) {
    System.out.println("testRequestHeader, Accept-Language: " + al);
    return "zwj";
}
```

@CookieValue

绑定请求中的 Cookie 值

```
 @RequestMapping("/testCookieValue")
public String testCookieValue(@CookieValue("JSESSIONID") String sessionId) {
    System.out.println("testCookieValue: sessionId: " + sessionId);
    return "zwj";
}
```

将 POJO 对象绑定为请求参数值

按请求参数名和 POJO 属性名进行自动匹配，自动为该对象填充属性值。

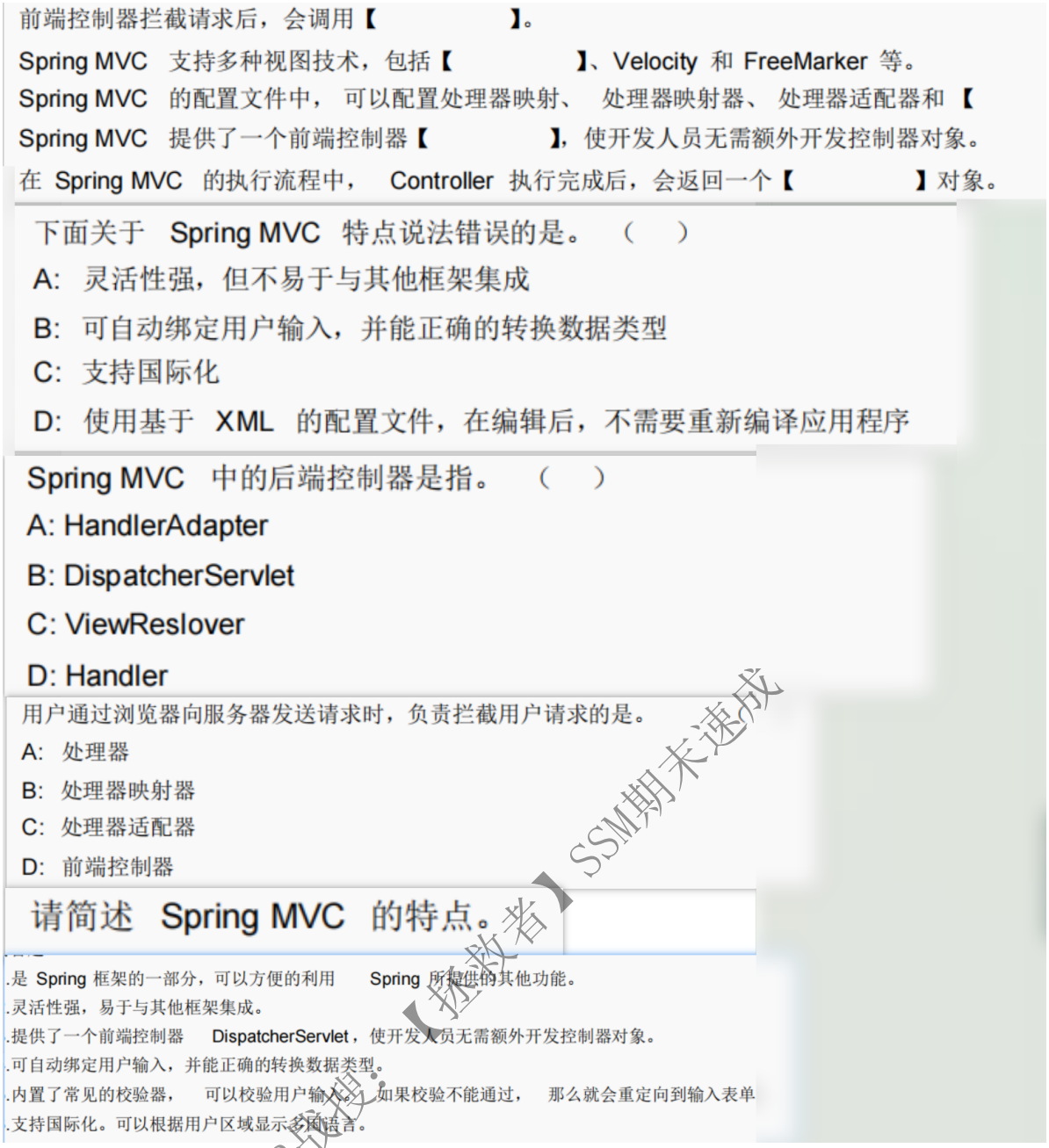
```
    @RequestMapping("/testPojo")
    public String testPojo(@RequestBody User user) {  使用@RequestBody接收JSON数
据:
        System.out.println("testPojo: " + user);
            return "zwj";
}
```

【拯救者】SSM期末速成
B战搜:

【拯救者】Ep22_SpringMVC基础-习题

B战搜: 【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)



B战搜：【拯救者】SSM期末速成

SpringMVC提供的核心组件包括前端控制器DispatchServlet）、请求处理映射器（HandlerMapping）、请求处理适配器（Handler Adapter）、视图解析器(View Resolver) 、动作处理器（Controller）和模型视图对象（ModelAndView） 后端控制器handler

下面关于<load-on-startup > 元素说法错误的是？ (c)

A.如果<load-on-startup > 元素的值为1,则在应用程序启动时会立即加载该Servlet

B.如果<load-on-startup-元素不存在,则应用程序会在第一个servlet请求时加载该Servlet

C.如果<load-on-startup-元素的值为1,则在应用程序启动时会延迟加载该Servlet

D.<load-on-startup-元素是可选的

如果< load-on-startup>元素的值为1，则在应用程序启动时会立即加载该Servlet

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

注解@RequestMapping可以标注在（AB）

A.一个类上

B.个方法上

C.一个属性上

D.以上都可以

请简述Spring MVC框架的工作执行流程

MVC框架的工作执行流程如下：客户端发送请求至DispatcherServlet，DispatcherServlet通过处理器映射器找到对应的Controller进行处理，Controller处理请求并返回ModelAndView对象，DispatcherServlet将ModelAndView传递给视图解析器解析为具体的视图对象，最终响应给客户端展示。在整个过程中，可以通过拦截器、异常处理器等组件进行拦截、处理和增强

【拯救者】Ep23_数据绑定和RESTful支持

B战搜：【拯救者】SSM期末速成

数据绑定

常用的数据绑定类型

1.基本数据类型　已讲
2.包装类
3.数组
4.对象　（表单的name值与对象的属性名对应即可完成对象的封装）　已讲
5.集合(List , Set , Map)
6.JSON　　已讲

```
    //包装类
    @RequestMapping(value = "/packageType")
    @ResponseBody
    public String packageType(@RequestParam(value = "id") Integer id){
        return "id:"+id;
    }
    //数组 自动绑定
    @RequestMapping(value = "/arrayType")
    @ResponseBody
    public String arrayType(String[] name){
        StringBuffer stringBuffer=new StringBuffer();
        for(String str:name){
            stringBuffer.append(str).append(" ");
        }
        return stringBuffer.toString();
    }
    //集合 不可以在参数中直接写List<Test>，要在一个bean中把list作为成员，才可以使用list绑定实体。
    UserVO类 里面有个List<User> users;
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
    @RequestMapping("/test")
     @ResponseBody
    public String test(UserVO userVo) {
        List<User> users=userVo.getUsers();
        return "ok";
    }
```

静态资源的处理

web应用部署到tomcat服务器上，服务器是先加载全局的`web.xml`，即加载我们自己web项目中的`web.xml`，当我们加`url-pattern`配置成`<url-pattern>/</url-pattern>`,导致静态资源也被我们自定义的dispatcherServlet所拦截

解决:

```
1. 手动排除法:
    <servlet-mapping>
        <servlet-name>default</servlet-name>     它是直接访问
        <url-pattern>*.jpg</url-pattern>
    </servlet-mapping>
2. 在springmvc的配置文件中加入<mvc:default-servlet-handler/>
3. 在springmvc配置文件中手动指定静态资源路径
    <mvc:resources location="/css/" mapping="/css/**" />
    <mvc:resources location="/img/" mapping="/img/**" />
    <mvc:resources location="/js/" mapping="/js/**" />
    <mvc:resources location="/plugins/" mapping="/plugins/**" />
```

自定义数据绑定

有些特殊类型的参数无法进行直接转换，例如日期数据就需要自定义转换器(Converter)或格式化(Formatter)来进行数据绑定。

Converter接口

```
    public interface Converter<S,T>{   S表示需要转换的类型，T表示目标类型
        T conver(S source);
    }
```

```
public class DataConverter implements Converter<String, Date> {
    //定义日期格式
    private String datePattern = "yyyy-MM-dd HH:mm:ss";

    @Override
    public Date convert(String source) {
        SimpleDateFormat sdf = new SimpleDateFormat(datePattern);
        try {
            return sdf.parse(source);
        }catch(ParseException e) {
            throw new IllegalArgumentException("无效格式,请使用这种格式:"+datePattern);
        }
    }

}
```

1. 修改spring-config.xml配置文件

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
<!-- 配置自定义类型转换器 -->
<bean id = "conversionService"
    class="org.springframework.context.support.ConversionServiceFactoryBean">
        <!-- 将自定义的转换器注入 -->
    <property name="converters">
        <set>
            <bean class="com.  .convert.DataConverter"/>
        </set>
    </property>
</bean>
<!-- 装配自定义类型转换器 -->
<mvc:annotation-driven conversion-service="conversionService"/>
```

指定自定义的转换器

使用名为conversionService的自定义类型转换器

Formatter接口

Formatter接口与Converter接口完成的功能类似。区别在于Converter接口可将任意一个类型的数据，转换成另一个类型的数据，而Formatter接口只能将一个String类型的数据转换成另一个类型的数据。

```
public class DateFormatter implements Formatter<Date> {
    //日期格式
    String datePattern = "yyyy-MM-dd HH:mm:ss";
    //声名 SimpleDateFormat对象
    private SimpleDateFormat simpleDateFormat;
    @Override
    //返回目标对象的字符串
    public String print(Date date, Locale locale) {
        return new SimpleDateFormat().format(date);
    }

    @Override
    //将字符串转换成日期型
    public Date parse(String source, Locale arg1) throws ParseException {
        simpleDateFormat = new SimpleDateFormat(datePattern);
        return simpleDateFormat.parse(source);
    }

}
```

https://blog.csdn.net/weixin_44949462

1. 修改spring-config.xml配置文件

```
<!-- 配置自定义类型转换器 -->
<bean id = "conversionService"
    class="org.springframework.format.support.FormattingConversionServiceFactoryBean">
        <!-- 将自定义的转换器注入 -->
    <property name="formatters">
        <set>
            <bean class="com.  .convert.DateFormatter"/>
        </set>
    </property>
</bean>
<!-- 装配自定义类型转换器 -->
<mvc:annotation-driven conversion-service="conversionService"/>
```

【拯救者】SSM期末速成

【拯救者】Ep24_数据绑定和RESTful支持-习题

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

在使用集合数据绑定时，后台方法中不支持直接使用集合形参进行数据绑定，所以需要使用包装 POJO 作为形参，然后在包装 POJO 中包装一个【 】属性。

使用包装 POJO 类型数据绑定时， 如果前端条件参数是包装类中 POJO 的子属性， 则参数名必须为【 】。

当前端请求的参数比较简单时，可以在后台方法的形参中直接使用 Spring MVC 提供的【 】类型进行数据绑定。

如果将订单和用户的所有查询条件都封装在一个简单 POJO 中，显然会比较混乱， 这时就可以考虑使用【 】类型的数据绑定。

在使用 POJO 类型数据绑定时，前端请求的参数名必须与要绑定的 POJO 类中的【 】一样，这样才会自动将请求数据绑定到 POJO 对象中，否则后台接收的参数值为 【 】。

以下有关 Spring MVC 数据绑定中集合数据绑定的说法正确的是。 （ ）

A: 批量删除用户操作时，前端请求传递过来的参数就会包含多个相同类型的数据，此时可以采用数组类型数据绑定的形式

B: 使用集合数据绑定需要后台方法中定义一个集合类型参数介绍绑定前端请求参数

C:绑定数组与绑定集合页面传递的参数相同，只是后台接收方法的参数不同

D: 在使用集合数据绑定时，后台方法中不支持直接使用集合形参进行数据绑定

@RequestParam 注解中可以省略属性名称的是。 （ ）

A: defaultValue

B: value

C: name

D: required

下面关于包装 POJO 类型数据绑定的说法正确的是。 （ ）

A: 如果查询条件参数是包装类的直接基本属性，则参数名直接用对应的属性名

B: 如果查询条件参数是包装类的直接基本属性，则参数名必须使用对应的 “对象.属性名 ”

C: 如果查询条件参数是包装类中 POJO 的子属性，则参数名必须为属性名

D: 如果查询条件参数是包装类中 POJO 的子属性，则参数名必须为 “对象 .子属性 .属性值 ”的形式

下面不属于 @RequestParam 注解类型属性的是。 （ ）

A: id

B: value

C: name

D: required

B战搜：【拯救者】SSM期末速成

【拯救者】Ep25_拦截器

拦截器概述

类似于Servlet开发中的过滤器Filter,**用于对处理器进行预处理和后处理。**

拦截器的使用

```
SpringMVC中拦截器的使用
        配置web.xml 和 springmvc-config.xml 文件
        编写一个拦截器
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
package com.wuji.interceptor;

import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.ModelAndView;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 * @Author 编程张无忌
 * @Version 1.0
 */

public class MyInterceptor implements HandlerInterceptor {

    //在请求处理的方法之前执行
    //如果返回true执行下一个拦截器
    //如果返回false就不执行下一个拦截器
    public boolean preHandle(HttpServletRequest httpServletRequest,
HttpServletResponse httpServletResponse, Object o) throws Exception {
        System.out.println("------------处理前------------");
        String username=httpServletRequest.getParameter("username");
        if(username != null&&username.length()>0){
            return true;
        }
        //  并转发到登录页面
        httpServletRequest.getRequestDispatcher("/index.jsp")
                .forward(httpServletRequest, httpServletResponse);
        return false;
    }

    //在请求处理方法执行之后执行  该方法会在控制器方法调用之后，且解析视图之前执行
    public void postHandle(HttpServletRequest httpServletRequest,
HttpServletResponse httpServletResponse, Object o, ModelAndView modelAndView)
throws Exception {
        System.out.println("------------处理后------------");
    }

    //在dispatcherServlet处理后执行 该方法会在整个请求完成，即视图渲染结束之后执行
    public void afterCompletion(HttpServletRequest httpServletRequest,
HttpServletResponse httpServletResponse, Object o, Exception e) throws Exception
{
    }
}
```

在springmvc的配置文件中配置拦截器

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
<!--关于拦截器的配置-->
<mvc:interceptors>
    <mvc:interceptor>
            <!--/** 包括路径及其子路径-->
            <!--/admin/* 拦截的是/admin/add等等这种 , /admin/add/user不会被拦截-->
            <!--/admin/** 拦截的是/admin/下的所有-->
        <mvc:mapping path="/login"/>
             <!--bean配置的就是拦截器-->
        <bean class="com.wuji.interceptor.MyInterceptor"/>
    </mvc:interceptor>
</mvc:interceptors>
```

编写一个Controller，接收请求

```
 //测试拦截器的控制器
@Controller
public class InterceptorController {

    @RequestMapping("/interceptor")
    @ResponseBody
    public String testFunction(@RequestParam("name") String name) {
        System.out.println("控制器中的方法执行了");
        return "hello";
    }
}
```

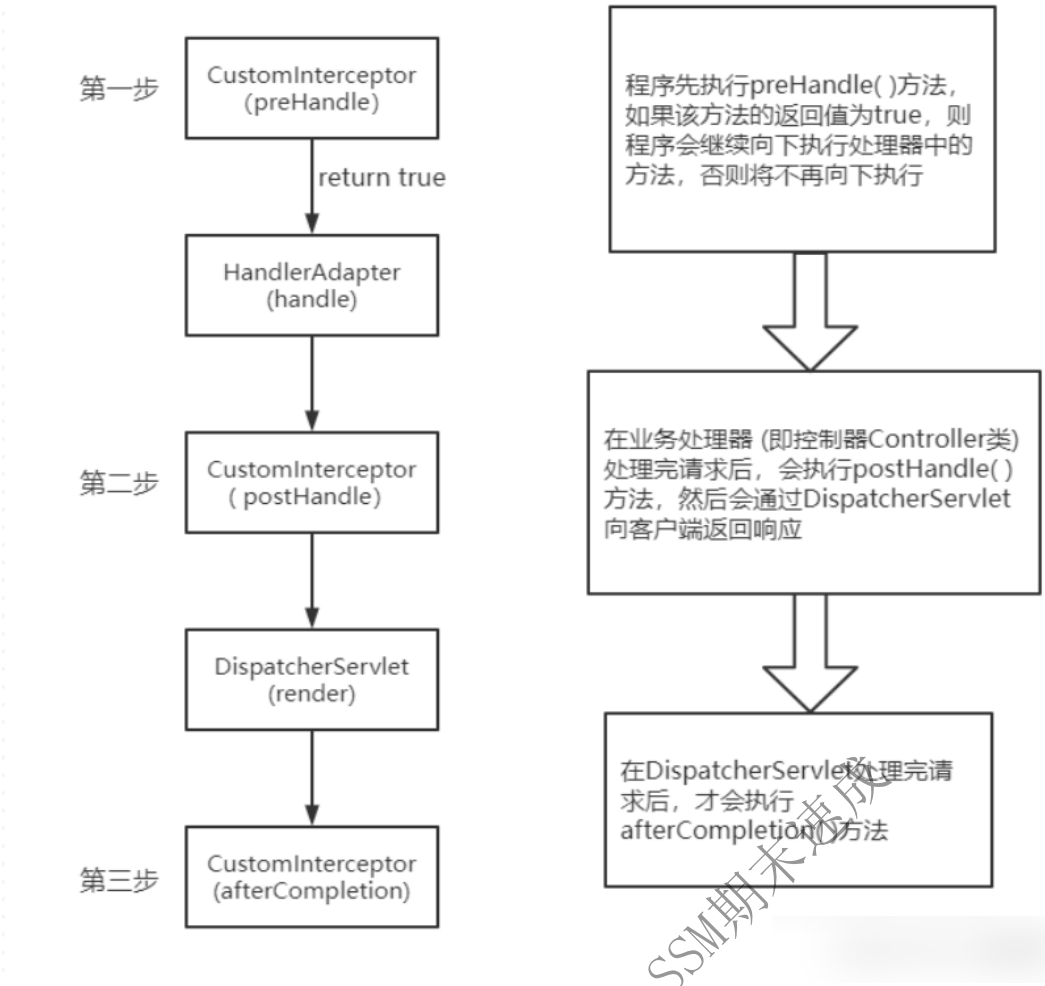
B战搜：【拯救者】SSM期末速成

单个拦截器的执行流程

首先会执行拦截器类中的preHandle()方法，如果该方法的返回值为true，则程序会继续向下执行处理器中的方法，否则将不再向下执行；在业务处理器（即控制器Controller类）处理完请求后，会执行postHandle()方法，然后会通过DispatcherServlet向客户端返回响应；在DispatcherServlet处理完请求后，才会执行afterCompletion()方法

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)



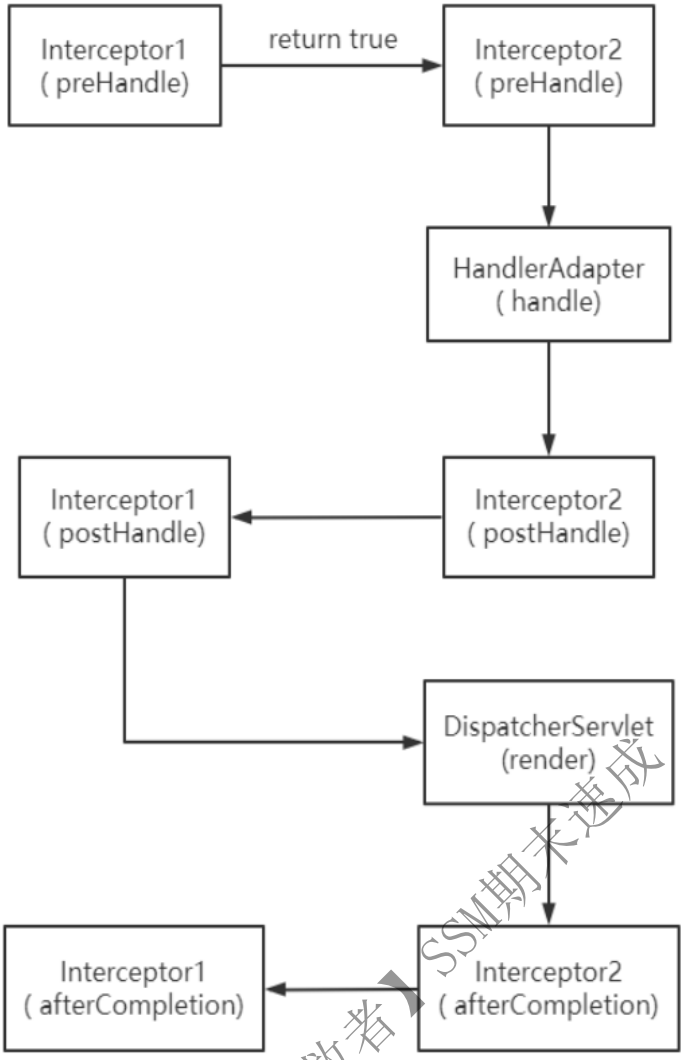
多个拦截器的拦截流程

它们的 preHandle( )方法会按照配置文件中拦截器的配置顺序执行，
而它们的 postHandle( )方法和 afterCompletion( )方法则会按照配置顺序的反序执行。

B战搜：【拯救者】SSM期末速成

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)



【拯救者】Ep26_拦截器-习题

B战搜:【拯救者】SSM期末速成

B战搜: 【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

Spring MVC 单个拦截器执行顺序中， 在【 】处理完请求后， 才会执行 afterCompletion() 方法。

Spring MVC 中的拦截器（ Interceptor ）类似于 Servlet 中的【 】，它主要用于拦截用户请求并作相应的处理。

用于如果没有登录系统而直接访问主页面，拦截器会将请求拦截，并转发到【 】。

当有多个拦截器同时工作时，它们的【 】方法会按照配置文件中拦截器的配置顺序执行。

如果拦截器类中的 preHandle()方法的返回值为【 】，则程序会继续向下执行处理器中的方法。

以下有关 Spring MVC 配置文件中拦截器的配置说法错误的是。 （ ）

A: 要使用 Spring MVC 中拦截器，要先自定义拦截器还需要在配置文件中进行配置。

B: <mvc:interceptors> 元素用于配置一组拦截器，其子元素 <bean>中定义的是指定路径的拦截器。

C: <mvc:interceptors> 元素中可以同时配置多个 <mvc:interceptor> 子元素。

D: <mvc:exclude - mapping>元素用于配置不需要拦截的路径请求。

下面不属于拦截器类中的方法的是。 （ ）

A: preHandler()

B: postHandle()

C: afterCompletion()

D: afterpletion()

以下哪个方法可以定义 Spring MVC 中的拦截器。 （ ）

A: 继承 HandlerInterceptor

B: 实现 WebRequestInterceptor

C: 实现 HandlerInterceptorAdapter

D: 继承 WebRequestInterceptor

下列关于拦截器的执行流程说法错误的是。 （ ）

A：程序首先会执行拦截器类中的 preHandle()方法。

B：如果 preHandle()方法的返回值为 true，则程序会继续向下执行处理器中的方法，否则将不再向下执行。

C：在业务处理器（即控制器 Controller 类）处理完请求后，会执行 preHandle()方法。

D：在 DispatcherServlet 处理完请求后，才会执行 afterCompletion()方法。

请简述拦截器 HandlerInterceptor 接口中的方法并做说明。

1、preHandler()方法：该方法会在控制器方法前执行，其返回值表示是否中断后续操作。当其返值为 true 时，表示继续向下执行；当其返回值为 false 时，会中断后续的所有操作。

2、postHandle()方法：该方法会在控制器方法调用之后，且解析视图之前执行。可以通过此方对请求域中的模型和视图做出进一步的修改。

3、afterCompletion()方法：该方法会在整个请求完成，即视图渲染结束之后执行。可以通过此方法实现一些资源清理、记录日志信息等工作。

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

【拯救者】Ep27_文件上传下载(上)

MultipartResolver概述

上传文件的组件,`StandardServletMultipartResolver`和`CommonsMultipartResolver`,前者采用Servlet3.0标准的上传方式,后者则使用了Apache的commons-fileupload

CommonsMultipartResolver实现方式

```
        <dependency>
            <groupId>commons-fileupload</groupId>
            <artifactId>commons-fileupload</artifactId>
            <version>1.4</version>
        </dependency>
```

```
 @RequestMapping("/play")
    @ResponseBody
    public String play1(MultipartFile file, HttpServletRequest req) throws
IOException {
        String format = simpleDateFormat.format(new Date());  //获取今天的日期
        String realpath = req.getServletContext().getRealPath("/img") +format;
//获取Tomcat中的临时储存文件
        File folder = new File(realpath);
        if(!folder.exists()){   //检查目录是否存在
            folder.mkdirs();    //目录不存在进行创建
        }
        String oldname = file.getOriginalFilename(); //得到上传时的文件名

        file.transferTo(new File(folder,oldname)); //将前端传来的数据保存指定名字保存
在指定文件夹下
        String url = req.getScheme()+ "://"+req.getServerName()
+":"+req.getServerPort() +"/img" +format +name;
        return url;


    }
```

B战搜:【拯救者】SSM期末速成

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
<bean class="org.springframework.web.multipart.commons.CommonsMultipartResolver"
id="multipartResolver">
     <!--默认的编码-->
    <property name="defaultEncoding" value="UTF-8"/>
    <!--上传的总文件大小-->
    <property name="maxUploadSize" value="1048576"/>
    <!--上传的单个文件大小-->
    <property name="maxUploadSizePerFile" value="1048576"/>
    <!--内存中最大的数据量，超过这个数据量，数据就要开始往硬盘中写了-->
    <property name="maxInMemorySize" value="4096"/>
    <!--临时目录，超过 maxInMemorySize 配置的大小后，数据开始往临时目录写，等全部上传完成
后，再将数据合并到正式的文件上传目录-->
    <property name="uploadTempDir" value="file:///E:\\tmp"/>
</bean>
```

【拯救者】Ep28_文件上传下载(下)

StandardServletMultipartResolver实现方式

```
    <bean
class="org.springframework.web.multipart.support.StandardServletMultipartResolve
r" id="multipartResolver"></bean>
```

将上传文件的参数配置放在web.xml的标签中

```
 <multipart-config>
            <!--文件保存的临时目录，这个目录系统不会主动创建-->
<!--            <location>D:\\temp</location>-->
            <!--上传的单个文件大小-->
            <max-file-size>1048576</max-file-size>
            <!--上传的总文件大小-->
            <max-request-size>1048576</max-request-size>
            <!--这个就是内存中保存的文件最大大小-->
            <file-size-threshold>4096</file-size-threshold>
  </multipart-config>
```

上传多个文件

```
  MultipartFile file 换成 MultipartFile 【】 file  即可  然后去遍历 file
```

文件下载

```
<h1>文件下载</h1>
<a href="${pageContext.request.contextPath}/file/download?
fileName=a.txt">aa.txt</a>
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
@Controller
@RequestMapping("file")
public class FileController {

    // 用来处理文件下载 请求对应响应输出流
    @RequestMapping("download")
    public void download(String fileName, HttpServletRequest request,
HttpServletResponse response) throws IOException {
        System.out.println("下载文件的名称：" + fileName);
        // 1. 根据下载相对目录获取下载目录在服务器部署之后绝对目录
        String realPath =
request.getSession().getServletContext().getRealPath("/down");
        // 2. 通过文件输入流读取文件
        FileInputStream is = new FileInputStream(new File(realPath, fileName));
        // 3. 获取响应输出流
        response.setContentType("text/plain;charset=UTF-8");
        // 4. 附件下载 attachment 附件 inline 在线打开(默认值)
        response.setHeader("content-disposition", "attachment;fileName=" +
fileName);
        // 5. 处理下载流复制
        ServletOutputStream os = response.getOutputStream();
        int len;
        byte[] b = new byte[1024];
        while(true){
            len = is.read(b);
            if(len == -1) break;
            os.write(b, 0, len);
        }
        // 释放资源
        os.close();
        is.close();
    }
}
```

B战搜:【拯救者】SSM期末速成

【拯救者】Ep29_文件上传下载-习题

B战搜:【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

下面关于 MultipartFile 接口中说法错误的是。 （ ）

A: getOriginalFilename() 用于获取上传文件的初始化名。

B: getSize() 用于获取上传文件的大小，单位是 KB 。

C: getInputStream() 用于读取文件内容，返回一个 InputStream 流。

D: transferTo(File file) 用于将上传文件保存到目标目录下。

下面属于 CommonsMultipartResolver 属性的是。（ ）

A: getContentType

B: getInputStream

C: isEmpty

D: defaultEncoding

下面关于文件上传表单说法错误的是。 （ ）

A: form 表单的 method 属性设置为 post

B: form 表单的 method 属性设置为 get

C: form 表单的 enctype 属性设置为 multipart/form -data

D: 提供 <input type="file" name="filename" /> 的文件上传输入框

下面关于文件下载方法内容描述错误的是。 （ ）

A：响应头信息中的 MediaType 代表的是 Interner Media Type（即互联网媒体类型），也叫做 MIME 类型。

B： MediaType.APPLICATION_OCTET_STREAM 的值为 application/octet -stream，即表示以二进制流的形式下载数据。

C： HttpStatus 类型代表的是 Http 协议中的状态。

D： HttpStatus.OK 表示 500，即服务器已成功处理了请求。

B战搜【拯救者】SSM期末速成

下面不属于文件解析器类 CommonsMultipartResolver 属性的是。（

A： maxUploadSize

B： maxInMemorySize

C： defaultEncoding

D： lazy

HttpStatus 类型代表的是 Http 协议中的状态，示例中的 HttpStatus.OK 表示【务器已成功处理了请求。

请简述文件上传时表单需要满足的 3 个条件。

1.form 表单的 method 属性设置为 post；

2.form 表单的 enctype 属性设置为 multipart/form -data；

3.提供 <input type="file" name="filename" /> 的文件上传输入框。

B战搜： 【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

http常见状态码有哪些?

一、1开头的状态码(信息类)

100，接受的请求正在处理，信息类状态码

二、2开头的状态码(成功类)

2xx(成功)表示成功处理了请求的状态码
200(成功)服务器已成功处理了请求。

三、3开头的状态码(重定向)

3xx(重定向)表示要完成请求，需要进一步操作。通常这些状态代码用来重定向。
301，永久性重定向，表示资源已被分配了新的 URL
302，临时性重定向，表示资源临时被分配了新的 URL
303，表示资源存在另一个URL，用GET方法获取资源
304，(未修改)自从上次请求后，请求网页未修改过。服务器返回此响应时，不会返回网页内容

四、4开头的状态码(客户端错误)

4xx(请求错误)这些状态码表示请求可能出错，妨碍了服务器的处理
400(错误请求)服务器不理解请求的语法
401表示发送的请求需要有通过HTTP认证的认证信息
403(禁止)服务器拒绝请求
404(未找到)服务器找不到请求网页

五、5开头的状态码(服务器错误)

5xx(服务器错误)这些状态码表示服务器在尝试处理请求时发生内部错误。这些错误可能是服务器本身的错误，而不是请求的错误
500，(服务器内部错误)服务器遇到错误，无法完成请求
503，表示服务器处于停机维护或超负载，无法处理请求

B战搜：【拯救者】SSM期末速成

【拯救者】Ep30_题型补充

简答

Spring注解中@Autowired和@Resource的区别

@Autowired属于spring，而@resource属于J2EE
@Autowired默认按类型装配，默认情况下必须要求依赖对象必须存在，如果要允许null值，可以设置它的required属性为false
@Resource，默认安装名称进行装配，名称可以通过name属性进行指定，如果没有指定name属性，当注解写在字段上时，默认取字段名进行安装名称查找，如果注解写在setter方法上默认取属性名进行装配。当找不到与名称匹配的bean时才按照类型进行装配。但是需要注意的是，如果name属性一旦指定，就只会按照名称进行装配

请简述AOP的原理？AOP有何应用价值

原理：
AOP，即面向切面编程。AOP 代理其实是由 AOP 框架动态生成的一个对象，该对象可作为目标对象使用。AOP 代理包含了目标对象的全部方法，但 AOP 代理中的方法与目标对象的方法存在差异：AOP 方法在特定切入点添加了增强处理，并回调了目标对象的方法。

应用价值：
通过 AOP 来处理一些具有横切性质的系统级服务，如事务管理、安全检查、缓存、对象池管理等，AOP 已经成为一种非常常用的解决方案。降低了系统耦合，减少了重复代码，提高了可维护性

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

Spring框架中，什么是控制反转？什么是依赖注入？有何区别?

控制反转：将组件间的依赖关系从程序的内部交给了IOC容器管理，也就是将创建bean的过程给反转了。传统的方式需要自己创建实例，现在在需要时只需要在IOC中获取即可。

依赖注入：将组建的依赖通过外部的参数或其他的形式注入。

区别： 依赖注入是从应用程序的角度在描述，可以把依赖注入 ；而控制反转是从容器的角度在描述，由容器反向的向应用程序注入应用程序所需要的外部资源。

事务的四个特性和每种特性的说明

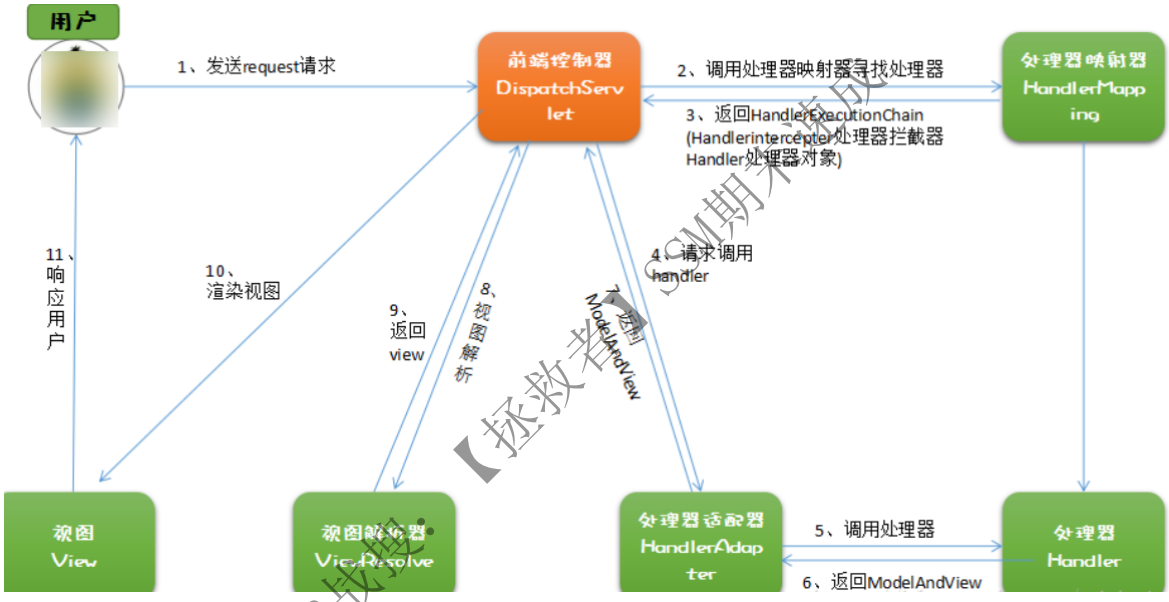
原子性：指事务包含的操作要么全部成功，若有一个失败，则全部回滚。操作失败对数据库不会产生任何影响。即要么全都执行，要么都不执行。

一致性：指事务必须使数据库从一个一致性的状态换到另一个一致性的状态。

隔离性：多事务并发时，一个事事务的执行不影响其他事务。

持久性：已经提交的事务对数据库的修改应该永久保存在数据库中。

SpringMVC工作原理



1、用户发送请求到前端控制器（DispatcherServlet）。
2、前端控制器请求处理器映射器（HandlerMapping）去查找处理器（Handler）。
3、找到以后处理器映射器（HandlerMappering）向前端控制器返回执行链（HandlerExecutionChain）。
4、前端控制器（DispatcherServlet）调用处理器适配器（HandlerAdapter）去执行处理器（Handler）。
5、处理器适配器去执行Handler。
6、处理器执行完给处理器适配器返回ModelAndView。
7、处理器适配器向前端控制器返回ModelAndView。
8、前端控制器请求视图解析器（ViewResolver）去进行视图解析。
9、视图解析器向前端控制器返回View。
10、前端控制器对视图进行渲染。
11、前端控制器向用户响应结果

Spring控制器接受请求参数的常见方式有哪几种。

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

- 通过实体bean接收请求参数
- 通过处理方法的形参接收请求参数
- 通过HTTPServletRequest接收请求参数
- 通过@PathVariable接收URL中的请求参数
- 通过@RequestParam接收请求参数

编程

在SpringMVC框架中，通过AJAX请求http://localhost:8080/springmvcdemo/getUserList.action 向服务器发送查询用户列表请求，服务器端控制器返回包含所有用户对象User的集合对象userList，请根据此要求在横线处完善下述程序

```
@RequestMapping("  / ____springmvcdemo/getUserList.action_____")
public List< User> getUserList(){
        List<User> list = new ArrayList<User>();
        for (long i = 0; i <=10; i++) {
             __User user = new User_____________;
            user.setId(i);
            user.setName("ssm"+i);
            user.setBirthday(new Date());
            ____list.add(user)_____;
        }
          ___return list_______;
    }
```

使用mybatis框架完善以下基于user表的CRUD操作。user表所对应的实体类如下：

```
public class User {
    private int id;
    private String name;
    private int age;

    public User(String name,int age){
        this.name=name;
        this.age=age;
    }
    //省略setter和getter
}
```

请完善sql映射文件"com.wuji.mapper.UserMapper"的实现：

```
<insert id="insertUser" parameterType=" _com.wuji.pojo.User_____________">
    __INSERT INTO `user` (`id`, `name`, `age`) VALUES (#{ }, #{ name }, #{age
})__
</insert>

<delete id="deleteUser" parameterType=" ____Integer___________">
__DELETE FROM `user` WHERE (`id`=#{id})_

</delete>

<update id="updateUser" parameterType=" ____com.wuji.pojo.User___________ ">
```

B战搜：【拯救者】SSM期末速成

定期扫描侵权店铺和违规分享(小红S,xian鱼等)

```
__UPDATE `user` SET `name`=#{ name }, `age`=#{  age } WHERE (`id`=#{ id})_

</update>

<select id="selectUserById" parameterType="int" resultType="
___com.wuji.pojo.User_____________ ">
____SELECT * FROM `user` WHERE `id` = #{id}__
</select>

<select id="selectAllUsers"  resultType="___com.wuji.pojo.User_____________">
__SELECT * FROM `user` _
</select>
```

在核心配置文件中配置sql映射文件:

完善以下测试类中对user表的CRUD的操作:

```
public void insertUser(int id) {
        SqlSession session = sessionFactory.openSession();
        //实例化一个用户
__User user = User("wujiZ",18)___ ;
//添加上述用户到数据库
___session.insert(""com.wuji.mapper.UserMapper. insertUser",user)____;
                  session.delete("com.wuji.mapper.UserMapper.deleteUser",1);
//提交事务
_____session.commit()_________;
//关闭session
____session.close()_______;
}
```

B战搜:【拯救者】SSM期末速成

B战搜:【拯救者】SSM期末速成